Development of an Intelligent Assistant for Data Preparation Automation in Urban Transportation Management Systems

Thesis for the Award of the Degree of
MASTER
7M06105 — Computer Science and Engineering

By
Ramazan Seiitbek

Under the Supervision of
Aivar Sakhipov

School of Software Engineering
Astana IT University LLP
Astana, Kazakhstan
June 2026

Abstract

Urban transportation management systems (UTMS) depend on sensor and event data that are often incomplete, duplicated, or schema-inconsistent before any forecasting or anomaly workflow can run. This thesis designs, implements, and evaluates **Flowmatic**: an intelligent preparation and inference platform that automates quality assessment, routes prepared streams to modality-specific neural checkpoints, and publishes reproducible training artifacts.

Flowmatic implements a six-dimensional Data Quality Index (DQI), automated cleaning, and export in a containerised web stack. On clean Astana batch uploads the pipeline reports composite quality 100/100 in 406 ms; under controlled corruptions up to 20%, composite DQI recovers by up to 0.045 while invalid rows are removed. A smart-city workbench connects simulated traffic and weather sources, Manual/Auto routing over a documented registry, and medallion lake export.

Seven primary neural checkpoints are trained under a shared temporal hold-out protocol (70/15/15 split, three seeds). The severity classifier reaches macro-F1 $= 0.911 \pm 0.017$, exceeding sklearn baselines on the same held-out test partition; labels are rule-based synthetic attributes. TranAD achieves AUROC $= 0.84$ on injected window anomalies. A preparation ablation lowers PatchTST density RMSE from 0.90 to 0.58 after 20% corruption when corrupted cells are repaired. Rule-based Auto routing over 140 simulator events achieves 100% scenario-aligned policy consistency under documented oracles.

Artifacts are published on Hugging Face under the `pushthetempo/flowmatic-*` family. Batch API validation uses the Astana export only; live agency deployment and external routing labels remain future work.

Keywords: Data Preparation; Intelligent Transportation Systems; Smart Cities; Flowmatic; Model Routing; Time Series; Reproducibility

Dedication and acknowledgements

I thank the faculty and colleagues at Astana IT University who supported this research.

This research was supported by the Committee of Science of the Ministry of Science and Higher Education of the Republic of Kazakhstan (Grant No. BR24992852, project “Intelligent models and methods of Smart City digital ecosystem for sustainable development and the citizens’ quality of life improvement”).

I acknowledge the use of open traffic benchmarks, Hugging Face model hosting, and the Flowmatic platform developed during this thesis.

Student's declaration

I, Ramazan Seiitbek certify that the work embodied in this research/project work, carried out by me under the supervision of Aivar Sakhipov for the period of September 2024 to June 2026 at Astana IT University. The work has not been submitted for the award of any other degree/ diploma except where due acknowledgement has been made in the text.

I further declare that the material obtained from other sources has been duly acknowledged in the thesis.

SIGNED: ..... DATE: .....

Certificate by Supervisor

This is to certify that research work embodied in this thesis entitled "Development of an Intelligent Assistant for Data Preparation Automation in Urban Transportation Management Systems" submitted to Astana IT University, for the award of the degree of Master (7M06105 Computer Science and Engineering) has been carried out by Ramazan Seiitbek under my supervision at Astana IT University, Astana from September 2024 to June 2026.

To the best of my knowledge and belief, this work is original and has not been submitted so far in part or in full for the award of any degree or diploma of any University/ Institute.

SIGNED: ..... DATE: .....

List of Tables

List of Figures

Figure **Page**

Table of Contents

Chapter 1

Introduction

1.1 Relevance, Problem Statement, and Research Gap

Urban transportation management systems rely on heterogeneous data collected from sensors, CSV exports, API feeds, and streaming telemetry [1–3]. In practice, these sources often contain missing values, duplicated records, schema inconsistencies, invalid coordinates, delayed timestamps, and domain-specific anomalies. Such defects complicate forecasting, anomaly detection, classification, and other analytical workflows, because the reliability of downstream results depends directly on the quality and reproducibility of the preparation process [4].

Existing studies on automated preprocessing, intelligent transportation systems, and neural traffic analytics usually address these stages separately [5–11]. Data cleaning is often treated as a preliminary technical step, while model training, deployment, and monitoring are evaluated independently [12, 13]. As a result, transportation analysts and system operators may lack a unified mechanism for measuring data quality, applying consistent transformations, preserving preparation history, and connecting prepared datasets to downstream models.

The research problem addressed in this thesis is the lack of an auditable and reproducible workflow for automating data preparation in urban transportation management systems while preserving compatibility with downstream analytical models. This thesis investigates how such a workflow can be designed, implemented, and evaluated through

the **Flowmatic** platform. The dissertation builds on supporting articles developed during the master's programme (Appendix A)—on adaptive passenger-flow modelling, automated data preparation, cross-city federated forecasting, and event-driven traffic microservices—from which methodological and empirical insights were collected and applied here [14–18].

1.2 Aim and Objectives of the Research

The aim of this research is to develop and evaluate an intelligent software platform for automating data preparation in urban transportation management systems, with support for measurable data quality assessment, reproducible preprocessing, and integration with downstream analytical models.

To achieve this aim, the following objectives are defined:

1. To analyze existing approaches to data quality assessment, automated preprocessing, and model lifecycle management in intelligent transportation systems.
2. To design the architecture of a platform that supports batch and streaming data preparation workflows for urban transportation data.
3. To implement mechanisms for data quality assessment, cleaning, transformation, storage, and export.
4. To integrate the preparation workflow with a model registry and inference service for transportation-related analytical tasks.
5. To evaluate the proposed platform using data quality metrics, latency measurements, controlled corruption experiments, baseline comparisons, and routing validation.
6. To define the limitations of the proposed approach and identify directions for future deployment in real urban transportation environments.

The object of the research is the data preparation process in urban transportation management systems. The subject of the research is the set of methods and software mechanisms for automated quality assessment, preprocessing, routing, and preparation of transportation data for analytical models.

1.3 Scientific Novelty and Practical Significance

The scientific novelty of the research lies in the integrated evaluation of data preparation, quality assessment, and model-oriented processing within a single reproducible workflow for urban transportation data.

The main elements of novelty are:

1. An integrated preparation-to-inference lifecycle that combines data quality assessment, automated cleaning, export, model registry usage, and downstream inference in one workflow.
2. A six-dimensional Data Quality Index applied to urban transportation data and evaluated under controlled data corruption scenarios [15].
3. A reproducible evaluation protocol linking preparation quality, batch-processing latency, baseline comparisons, neural model performance, and routing validation.
4. A model-oriented routing mechanism that connects prepared traffic and sensor streams to task-specific analytical models [17].
5. Published artifacts and evaluation outputs that support reproducibility of the experimental results [13, 19].

The practical significance of the research is represented by the implemented Flowmatic platform, which provides authenticated data upload, automated quality assessment, cleaning, storage, export, smart-city simulation workflows, model registry integration, and inference support. The platform can be used as a research prototype for preparing transportation datasets and demonstrating how data preparation can be connected with downstream analytics in urban transportation management systems.

1.4 Thesis Structure

The thesis is organized as follows. Chapter 2 reviews related work on data quality, automated preprocessing, intelligent transportation systems, forecasting, anomaly detection, model registries, and reproducibility. Chapter 3 describes the research methodology, including the data quality assessment approach, preparation workflow, routing logic,

evaluation protocols, and research questions. Chapter 4 presents the implementation of the Flowmatic platform. Chapter 5 reports experimental results and discusses preparation quality, latency, baseline comparisons, neural evaluation, and routing validation. Chapter 6 summarizes the findings, limitations, and future research directions. Appendix A lists the author's prior publications; Appendix B documents reproducibility commands; Appendix C describes Astana dataset construction.

Chapter 2

Literature Review

2.1 Scope and Organisation

This chapter positions Flowmatic against four research streams: (i) data quality and automated preparation; (ii) urban ITS platforms, streaming, and digital-twin deployments; (iii) classical and neural time-series methods for traffic analytics; and (iv) MLOps registries and reproducible model lifecycles. The goal is not to catalogue every related paper but to show where prior work stops short of an *auditable preparation-to-inference workflow* with measured baselines, routing behaviour, and published checkpoints—the gap this thesis addresses at master's level.

2.2 Data Quality and Fitness for Use

Early information-systems research defines data quality as fitness for use rather than abstract correctness [1, 2]. Rahm and Do [20] classify cleaning problems (missing values, duplicates, inconsistencies) that appear routinely in transportation exports. Heinrich et al. [21] show that composite indices help operational teams prioritise dimensions when weights remain application dependent.

Flowmatic adopts six dimensions—completeness, consistency, accuracy, timeliness, uniqueness, and validity—with equal default weights. This follows composite-index practice but does *not* claim a new quality theory; the contribution is operational measurement tied to automated remediation and reported latency on urban traffic uploads.

2.3 Automated Preprocessing and AutoML Pipelines

Manual notebook pipelines are difficult to audit when analysts or models change [4]. Automated preprocessing adapts operations to schema and downstream tasks [5, 22]. AutoML systems such as auto-sklearn integrate hyperparameter search with preprocessing [6, 7, 23]. Mahasivam et al. [24] describe preparation-as-a-service back-ends for scalable ingestion.

These systems optimise model accuracy on static tables. They rarely expose six-dimensional diagnostics to transportation operators, connect preparation scores to streaming smart-city pipelines, or publish TorchScript checkpoints with routing metadata. Flowmatic therefore complements AutoML research with an ITS-facing batch and streaming prototype rather than competing on search efficiency.

2.4 Missing Data, Imputation, and Traffic Streams

Traffic streams are spatially distributed and time ordered [25, 26]. Variational and deep imputation models exploit temporal structure [27]. SAITS-style masked reconstruction is widely used for multivariate gaps [26]. Flowmatic deploys SAITS within its production portfolio and reports masked MSE on held-out windows (Chapter 5), separate from batch DQI cleaning.

2.5 Feature Engineering and Spatiotemporal Representation

Hand-crafted cyclical time features, rolling statistics, and graph adjacency remain standard precursors to deep traffic models [28–31]. Graph convolutional and adaptive embedding Transformers improve forecasting on road networks [10, 32–35]. Flowmatic materialises windowed tensors and adjacency for STGCN while keeping schema-level features auditable in batch uploads.

2.6 Forecasting Baselines and Neural Architectures

Classical forecasting—ARIMA/Box–Jenkins [36], exponential smoothing, and Prophet-style decompositions [37, 38]—remains the reference line for transportation agencies. Neural surveys emphasise that Transformers are not uniformly superior; linear decompositions can match or beat attention on many benchmarks [11, 39]. Patch-based Transformers [9], Informer-style long-sequence models [40], and LSTM baselines [41] motivate the deployed portfolio. Attention foundations [42] underpin the severity classifier.

The thesis reports naive last-value and seasonal naive RMSE on raw density, plus logistic regression and majority-class predictors for severity, on identical temporal splits. Neural scores are computed on z-score normalised windows; the baselines clarify scale and task difficulty rather than claiming leaderboard parity with published PatchTST results on ETT, which rely on different splits [9].

2.7 Anomaly Detection: Classical and Deep

Surveys distinguish point, contextual, and collective anomalies [43]. Classical methods include LOF [44] and Isolation Forest [45]. Deep reviews cover reconstruction and generative approaches [46–49]. Urban traffic management increasingly couples real-time anomaly screening with load balancing [50, 51].

Flowmatic deploys TranAD-style reconstruction on Astana windows. The thesis reports reconstruction RMSE and injected-window AUROC on held-out splits; municipal incident labels are out of scope, so reconstruction quality is not conflated with field-verified detection performance.

2.8 Classification and Tree Ensembles

Severity and incident typing historically relied on tree ensembles and SVMs [52–54]. The Transformer classifier slot provides a neural upper bound under class imbalance (Low 91%, High 7%, Medium 2% in the Astana export). Comparing against majority-class, logistic regression, and random forest baselines contextualises classifier performance on the *same* held-out temporal test partition.

2.9 Intelligent Transportation Systems and Event-Driven Platforms

Smart-city architectures combine sensing, edge processing, and analytics [3, 8]. Event-driven middleware supports publish/subscribe ITS pipelines [55, 56]. Digital-twin surveys describe city-scale coupling of simulation and live feeds [57]. Diffusion and graph recurrent models remain influential traffic forecasters [32, 58].

Flowmatic uses HTTP/WebSocket ingestion, queue workers, and a dedicated inference microservice—aligned with ITS practice but implemented as a research prototype, not a certified safety-critical deployment.

2.10 Explainability and Operator Trust

Transportation agencies require traceable diagnostics [59–62]. Flowmatic emphasises deterministic Insight Engine metrics and neural residuals; optional large-language-model narration is constrained to pipeline context and is not formally evaluated.

2.11 MLOps, Registries, and Reproducibility

MLflow [12] and MLOps guidance [13] standardise experiment tracking and model registry patterns. Hugging Face tooling democratises dataset and model publication [19]. Flowmatic adapts registry ideas with per-checkpoint metadata (modality, task, geo requirement, priority) and public Hub weights; the scientific claim is the **linked evaluation protocol**, not a new registry standard.

2.12 Comparative Positioning

Table 2.1 contrasts adjacent systems. Notebook ETL and AutoML optimise static tables; MLflow-centric stacks rarely integrate six-dimensional QC with live routing demos. None of the listed alternatives jointly report preparation stress tests, routing accuracy, and multi-task neural artifacts under one reproducible script bundle in this thesis.

Table 2.1: Related systems versus Flowmatic (high-level comparison).

System class	Prep QC	Streaming	Registry	Multi-task eval	Gap vs. Flowmatic
Notebook / pandas ETL	Ad hoc	Rare	No	Ad hoc	No unified routing or Hub publish
Auto-sklearn / AutoML	Yes	No	Partial	CV only	No ITS streaming or DQI dashboard
MLflow + microservices	Partial	Yes	Yes	Per model	No six-D QC + routing pilot in one study
Digital-twin platforms	Simulation	Yes	Varies	Scenario-based	Rarely publish neural prep baselines
Flowmatic (this thesis)	Yes (6-D)	Yes (demo)	Yes	Yes (8 slots)	Closes prep–route–eval loop

2.13 Research Gap and Thesis Positioning

Earlier publications by the author and collaborators addressed passenger-flow hybrids, preparation automation, and event-driven traffic microservices [14–17].

2.13.1 Comparable systems

auto-sklearn [6, 7] optimises preprocessing and model choice inside cross-validation on static tables. It reports accuracy and runtime but not six-dimensional QC traces tied to transportation uploads, streaming smart-city routing, or published TorchScript checkpoints with per-task metrics on identical temporal splits.

Flowmatic (prior platform paper) [15] documents preparation automation and a neural portfolio but does not, in that venue, report corruption stress tests with before/after DQI, sklearn baselines on the same held-out tail, or routing accuracy on labelled simulator events. This thesis closes that measurement gap while reusing the same engineering stack.

Prior work in the wider literature rarely connects three measurable layers in one master's study: (i) controlled preparation on corrupted traffic uploads with before/after DQI; (ii) downstream neural evaluation with simple baselines on identical temporal splits; and (iii) quantified routing over a modality-aware registry. Flowmatic targets that integration gap. It does **not** claim a new learning algorithm; the scientific contribution

is the **evaluation protocol and honest scope boundaries** (semi-synthetic Astana, batch API validation on one CSV family, neural multi-dataset training without claiming each dataset was uploaded through Nest).

2.14 Automated Machine Learning and Pipeline Search

Auto-sklearn and related systems treat preprocessing as a search problem inside cross-validation [6, 7, 23]. Hyperparameter optimisation surveys document the cost of nested model selection [6]. For transportation agencies, the limiting factor is often not AUC on a static table but **traceability**: which rows were dropped, which rules fired, and whether the same pipeline can be replayed after an analyst change. Flowmatic therefore emphasises logged DQI dimensions and export manifests rather than claiming superior AutoML search.

2.15 Traffic Forecasting Surveys and Graph Models

Diffusion convolutional recurrent networks [58] and attention graph models [32] remain influential baselines in traffic forecasting competitions. Recent surveys catalogue missing-data imputation and deep forecasting hybrids [25, 39]. Adaptive spatiotemporal Inception-style networks [35] and transformer embeddings for traffic [33, 34] illustrate rapid architectural churn. This thesis does not reproduce every leaderboard configuration; instead it trains a fixed portfolio under one temporal split to enable fair comparison across slots.

2.16 Digital Twins and Smart-City Operations

Digital-twin frameworks couple simulation, live feeds, and decision support [57]. They rarely publish both preparation QC under corruption and neural routing pilots in the same study. Flowmatic's smart-city module is a **research demonstrator**: simulated HTTP sensors, WebSocket UI updates, and medallion exports illustrate operational shape without claiming municipal certification.

2.17 Synthesis: Where Flowmatic Sits

Conceptually, prior systems form three clusters: disconnected notebooks; MLflow-style tracking without ITS QC; and Flowmatic's closed loop from upload → DQI → registry → inference → published metrics. The gap is **measurement linkage**, not a single new layer type.

Chapter 3 formalises DQI, routing, and metrics. Chapter 5 reports experiments and discussion. Appendix C documents dataset construction.

Chapter 3

Methodology

3.1 Overview

The methodology covers the **Flowmatic** preparation pipeline, the **production model registry**, and the **neural evaluation protocol** used in Chapter 5. Batch upload and smart-city deployment modes are illustrated in Chapter 4 (Figures 4.1 and 4.2).

3.2 Research Design

The study follows a **design–build–evaluate** pattern common in software engineering theses [4, 13]. Artifacts are: (1) the platform prototype; (2) published neural checkpoints; (3) evaluation tables binding RQ1–RQ4 to metrics. Internal validity comes from fixed seeds, held-out temporal test partitions, and scripted reproduction. External validity is limited by semi-synthetic Astana data and simulated streaming; threats are listed in Section 5.8.

3.3 Research Questions

The evaluation programme in Chapter 5 addresses the following research questions:

RQ1: Does the Flowmatic preparation workflow improve measurable data quality on corrupted urban traffic uploads (DQI before/after, rows dropped, latency)?

RQ2: Can preparation be operationalised as a reproducible batch service with reported end-to-end latency and quality scores on the Astana case study?

RQ3: How do deployed neural models perform against majority-class, logistic regression, random forest, naive, and seasonal baselines under a common temporal hold-out protocol?

RQ4: To what extent does rule-based Auto routing select scenario-aligned checkpoints on a labelled simulator corpus covering forecast, anomaly, classification, imputation, and weather profiles?

The research focuses on three connected aspects: whether data preparation improves measurable data quality, whether the prepared data can be operationalised through a reproducible platform workflow, and whether downstream model selection can be supported through a documented routing mechanism.

3.4 Evaluation Scope and Validity

Two validation layers must not be conflated. *Batch platform validation* is demonstrated on the Astana CSV export through the Nest ingestion API (latency, quality score, cleaning). *Neural training validation* uses Astana plus Hugging Face weather, ETT, and traffic-graph bundles prepared offline; those bundles are not all re-uploaded through the Nest API in this thesis. METR-LA and PEMS datasets are excluded from the evaluated dataset table (Table 5.3) because hold-out experiments were not completed for them in this study.

Experiments use semi-synthetic Astana data derived from an NDA-protected source (Appendix C; generator rules in the unpublished companion study [63], continuing the published microservices work [17]) with rule-based severity labels. The smart-city demo uses simulated feeds. Municipal incident labels, live agency deployment, and probabilistic forecast scores (CRPS) remain future work; injected-window TranAD AUROC is reported in Chapter 5.

3.5 Non-Functional Requirements

Operators require sub-second feedback on batch uploads (RQ2), authenticated multi-tenant organisation scoping, and export to both relational and object storage. Inference requires TorchScript latency compatible with smart-city polling (Section 5.6). Repro-

ducibility requires versioned manifests: dataset hashes, scaler statistics, metrics JSON, and Hub repository identifiers referenced in Table 4.2.

3.6 Data Quality Assessment

Flowmatic computes a weighted Data Quality Index (DQI) for each upload and pipeline run:

$$\mathrm{DQI} = \frac{\sum_{i=1}^{6} w_i Q_i}{\sum_{i=1}^{6} w_i}, \quad Q_i \in [0,1], \tag{3.1}$$

over completeness, consistency, accuracy, timeliness, uniqueness, and validity [1, 2, 21]. Completeness follows $C = 1 - |M|/|D|$; timeliness uses exponential decay $T = \exp(-\lambda \bar{a})$. Table 3.1 lists default weights $w_i = 1/6$; Table 5.11 in Chapter 5 reports sensitivity of the composite score at 20% corruption under alternative weightings. The batch quality module implements these dimensions through schema typing, duplicate detection, valid-range checks, and Z-score outlier flags.

Table 3.1: DQI dimension weights and score definitions (Flowmatic `QualityService`).

Dim.	**Criterion**	**Weight** w_i	**Score** Q_i
1	Completeness	1/6	$1 - \lvert M\rvert/\lvert D\rvert$
2	Consistency	1/6	Rule-pass rate on schema checks
3	Accuracy	1/6	Valid-range compliance
4	Timeliness	1/6	$\exp(-\lambda \bar{a})$ on arrival lag
5	Uniqueness	1/6	1− duplicate-row fraction
6	Validity	1/6	Type and constraint satisfaction

3.7 Preparation and Feature Windows

After quality assessment, workers apply imputation for missing numeric fields, duplicate removal, and type coercion. For neural training and inference, each sensor stream is windowed into tensors $\mathbf{X} \in \mathbb{R}^{L \times d}$ with $L = 48$ by default. Cyclical time encodings

$$f_{\sin}(t) = \sin(2\pi t/P), \quad f_{\cos}(t) = \cos(2\pi t/P) \tag{3.2}$$

and traffic-derived channels (density, speed, severity labels) are materialised in Phase 2 dataset bundles. Graph models additionally receive adjacency $\mathbf{A}$ for STGCN.

3.8 Adaptive Core Unit and Model Registry

3.8.1 Registry

Each production checkpoint m exposes metadata: model kind (anomaly, classifier, forecaster variants), dataset affinity, tasks, sensor kinds, optional geographic requirement, and priority score. The registry loads from local metadata files and the Hugging Face manifest (Table 4.2).

3.8.2 Routing policy

For sensor kind k and payload features $\mathbf{f}$, the router builds an event profile (modality, preferred task, geo flag). In **Manual** mode the operator selects model m^*. In **Auto** mode:

$$m^* = \arg\min_{m \in \mathcal{M}_k} \pi(m \mid \mathbf{f}), \tag{3.3}$$

where $\mathcal{M}_k$ filters incompatible modalities/tasks and π is a priority score (lower is better). Anomaly-capable tasks receive precedence when anomaly detection is enabled.

Algorithm 1 Rule-based model routing (Auto mode)

Input: sensor kind k, payload $\mathbf{f}$, registry $\mathcal{M}$, anomaly flag a
profile $\leftarrow$ CLASSIFY$(k, \mathbf{f})$
if a **and** $\exists m \in \mathcal{M}$ with task anomaly matching *profile* **then**
    return best-priority anomaly model (TranAD slot)
end if
$\mathcal{C} \leftarrow \{m \in \mathcal{M} : \mathit{modality}(m) = \mathit{profile.modality} \wedge \mathit{task}(m) \supseteq \mathit{profile.task}\}$
return $\arg\min_{m \in \mathcal{C}} \mathit{priority}(m)$

3.9 Neural Task Formulations

Each production model consumes a normalized window $\mathbf{X} \in \mathbb{R}^{L \times d}$ of length $L = 48$ and feature dimension d. Targets are denoted y; graph models use adjacency $\mathbf{A} \in \mathbb{R}^{N \times N}$ over N sensors.

3.9.1 Forecasting (PatchTST, iTransformer, DLinear, TimesBlock, STGCN)

Point forecasting minimizes squared error on a held-out temporal test split:

$$\mathcal{L}_{\text{forecast}} = \frac{1}{|\mathcal{T}_{\text{test}}|} \sum_{t \in \mathcal{T}_{\text{test}}} \|\hat{y}_t - y_t\|_2^2 . \tag{3.4}$$

DLinear decomposes each channel into trend and seasonal components via moving-average smoothing kernel $\mathbf{1}_k/k$ before applying linear projections [11]. PatchTST partitions $\mathbf{X}$ into patches of length p and applies channel-independent self-attention over patch tokens [9]. STGCN combines graph convolution on $\mathbf{A}$ with temporal convolution:

$$\mathbf{H}^{(l+1)} = \sigma\big(\mathbf{A}\mathbf{H}^{(l)}\mathbf{W}_{\text{g}}^{(l)}\big) * \mathbf{W}_{\text{t}}^{(l)}, \tag{3.5}$$

where $*$ denotes temporal convolution and σ is ReLU.

3.9.2 Anomaly detection (TranAD-style reconstruction)

Let f_θ reconstruct window $\mathbf{X}$. Anomaly score is reconstruction residual:

$$s(\mathbf{X}) = \frac{1}{Ld} \sum_{i=1}^{L} \sum_{j=1}^{d} \Big(X_{ij} - \hat{X}_{ij}\Big)^2 . \tag{3.6}$$

Training minimizes $\|\mathbf{X} - f_\theta(\mathbf{X})\|_F^2$ on normal traffic windows; elevated $s(\mathbf{X})$ flags corrupted streams.

3.9.3 Imputation (SAITS-style masked reconstruction)

Given mask $\mathbf{M} \in \{0,1\}^{L \times d}$ with mask ratio $\rho = 0.25$, the model predicts occluded values:

$$\mathcal{L}_{\text{impute}} = \frac{\sum_{i,j}(1 - M_{ij})(X_{ij} - \hat{X}_{ij})^2}{\sum_{i,j}(1 - M_{ij})}. \tag{3.7}$$

3.9.4 Severity classification (Transformer encoder)

For class set $\mathcal{C}$, the encoder outputs logits $\mathbf{z} = h_\phi(\mathbf{X})$ and probabilities $\pi_c = \text{softmax}(\mathbf{z})_c$. We report macro-F1:

$$F_1^{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \frac{2P_c R_c}{P_c + R_c}. \tag{3.8}$$

3.10 Neural Evaluation Methodology

Seven primary checkpoints and one auxiliary iTransformer speed slot share temporal splitting (Table 5.4 in Chapter 5), z-score standardisation fit on training windows only, and seed-controlled training. Objectives are defined in Section 3.9. Reported metrics use only the held-out 15% test partition; multi-seed aggregation is $\bar{m} \pm 1.96\, s/\sqrt{n}$ with $n = 3$.

3.10.1 Baseline protocols

Forecasting. Naive last-value and seasonal naive predictors operate on raw Astana density series aligned to the same temporal test windows as neural models, but without z-score normalisation. Reported naive RMSE values therefore illustrate scale and difficulty; they are not subtracted from neural RMSE on normalised tensors.

Classification. Majority-class, logistic regression, and random forest models are trained on the earliest 85% of chronologically ordered Astana rows and evaluated on the latest 15% test partition defined in Table 5.4—the same held-out temporal segment used for the Transformer severity checkpoint. Macro-F1 is the primary metric because classes are imbalanced [52].

Imputation. Mean imputation on z-score normalised windows provides an analytic masked-MSE baseline of 1.0 (predicting zero for missing values); SAITS is compared against this reference in Table 5.14.

Anomaly. TranAD is evaluated using reconstruction RMSE on held-out windows and AUROC against injected point anomalies (Table 5.16). External municipal incident labels and full LOF/Isolation Forest threshold-matched comparisons remain future work [44–46].

3.10.2 Preparation stress protocol

For RQ1, defects are injected into a 10 000-row Astana sample at rates $\{0, 1, 5, 10, 20\}\%$. Composite DQI is computed before and after the same cleaning rules used in production (duplicate removal, invalid coordinate rejection, missing numeric handling). Rows dropped are counted explicitly to distinguish record repair from record deletion.

3.10.3 Routing evaluation protocol

For RQ4, 140 simulator events carry *scenario-aligned* oracle checkpoint kinds: non-geolocated density forecast (PatchTST), geolocated graph forecast (STGCN), anomaly (TranAD), weather (TimesBlock), severity classification (Transformer), and sparse imputation (SAITS). The rule-based router in the production codebase is replayed offline. Generic-modality imputation checkpoints may serve traffic payloads when missingness exceeds the imputation threshold. Accuracy is the fraction of events for which the selected checkpoint kind equals the scenario oracle.

3.11 Insight Engine and Copilot

The Insight Engine composes deterministic analytics (hotspots, drift, correlation) with optional LLM narration constrained to retrieved pipeline context. Neural models provide primary diagnostics: reconstruction residuals (TranAD, Equation 3.6), imputation masks (SAITS), and classification logits (Transformer severity).

Chapter 4

Implementation

4.1 System Architecture

Flowmatic is implemented as a TypeScript monorepo: a NestJS backend with Prisma and PostgreSQL, a Vue 3 single-page application, a Python/Bun inference sidecar, and Docker Compose orchestration. Figure 4.3 shows PostgreSQL, MinIO object storage, RabbitMQ, the sensor simulator, the model-inference service, the Nest backend, and Nginx-served frontend. This architecture follows event-driven ITS practice [55, 56] while remaining a research prototype.

4.2 Batch Preparation Module

Clients upload CSV or JSON through authenticated REST endpoints. Files land in S3-compatible storage; workers parse records, compute the six-dimensional DQI, apply cleaning rules (imputation, duplicate removal, range checks), and persist summaries for the dashboard. Export adapters target relational databases, document stores, Hugging Face datasets, and flat files. Figure 4.1 corresponds to the measured upload experiment in Table 4.1.

4.3 Smart City Module

Organisation-scoped pipelines comprise four stages:

1. **Sources** — HTTP or WebSocket simulators and external connectors;

2. **Core unit** — validation, Manual/Auto routing, and inference invocation;
3. **Lake and export** — bronze, silver, and gold prefixes on object storage (Figure 4.4);
4. **Federated demo** — optional coordinator metadata for multi-site experiments.

WebSockets stream events to the operator interface; queue workers handle asynchronous export. The Astana Geospatial Demo exercises traffic and weather sources with Manual/Auto routing visible in Figures 4.6–4.8.

4.4 Model Inference Service

The inference microservice loads TorchScript checkpoints from local storage or Hugging Face Hub using the portfolio in Table 4.2. Requests specify checkpoint identity, window length L=48, and task-specific tensors (for example graph adjacency for the traffic-graph model). Latency benchmarks in Section 5.6 inform smart-city budgets.

4.5 Research Artifact Pipeline

Offline training prepares windowed datasets and runs the multi-seed evaluation suite. Each checkpoint ships metadata, scaler statistics, metrics JSON, and TorchScript exports. Publication under the `pushthetempo` Hub namespace supports third-party reproduction without relying on proprietary training notebooks.

4.6 User Interface

The web interface provides login, batch upload with quality dashboards, pipeline run history, and the smart-city workbench. Operators configure routing mode, inspect recent events, and open the workflow graph for the Astana simulator pipeline.

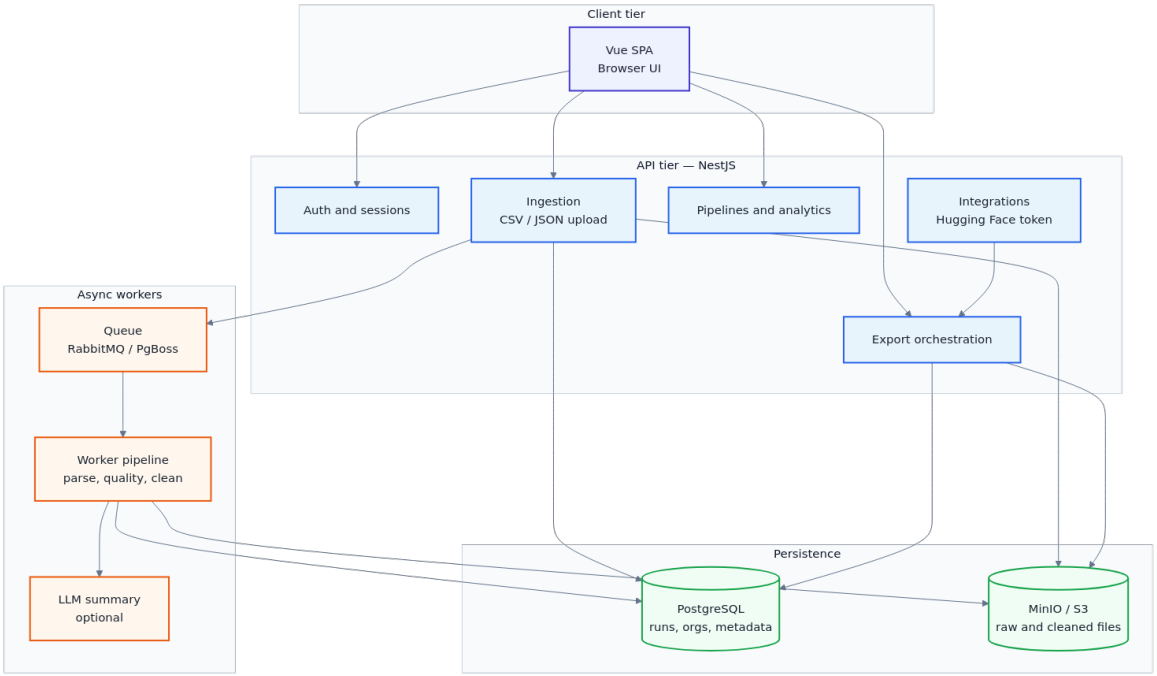

Figure 4.1: Batch upload pipeline (NestJS + Vue).

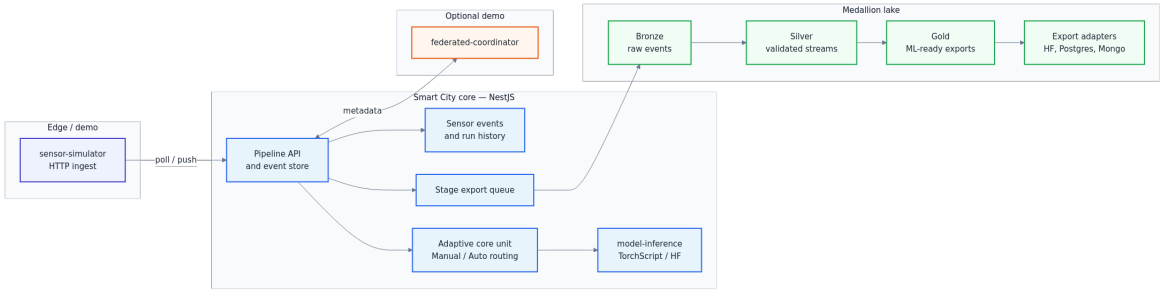

Figure 4.2: Smart-city pipeline with adaptive core unit and medallion lake.

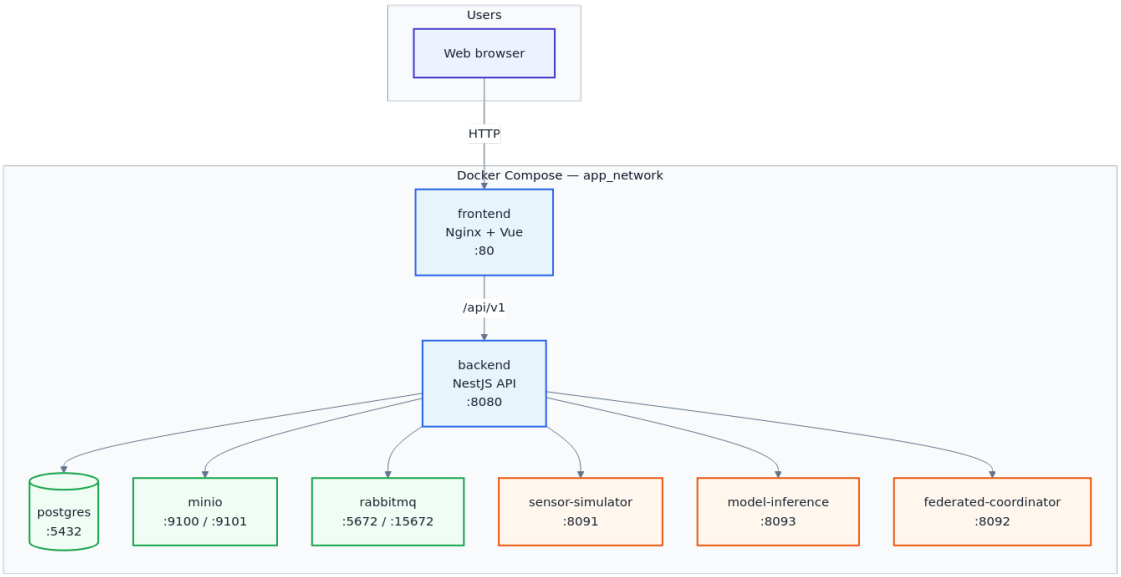

Figure 4.3: Docker Compose deployment topology.

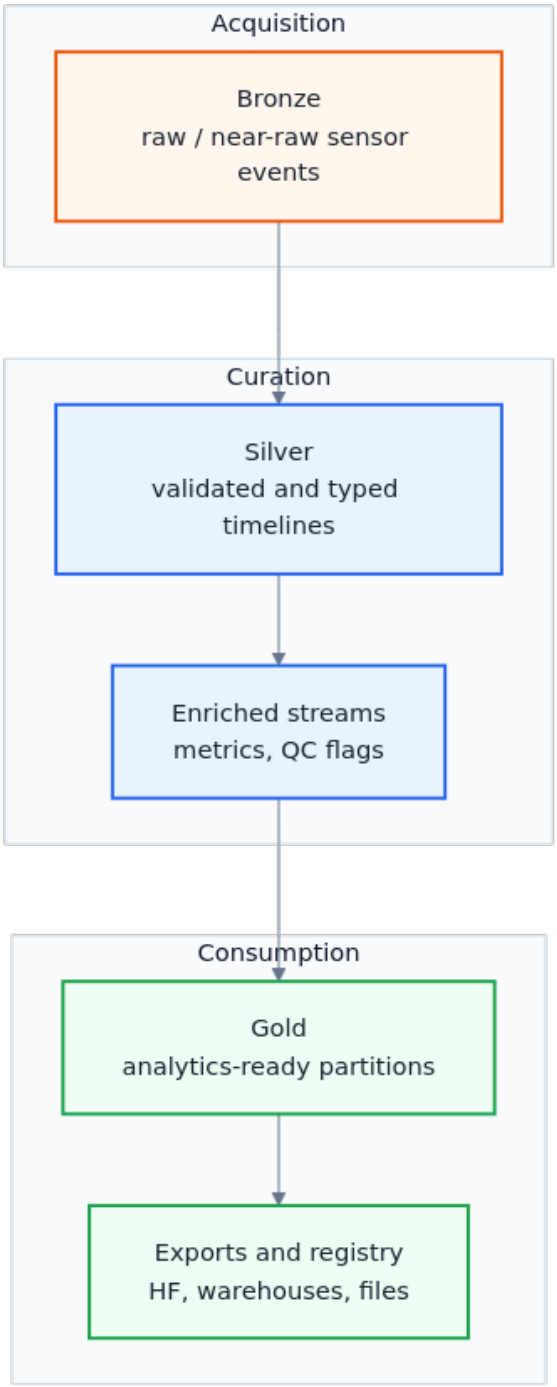

Figure 4.4: Medallion data lake layout (bronze, silver, gold).

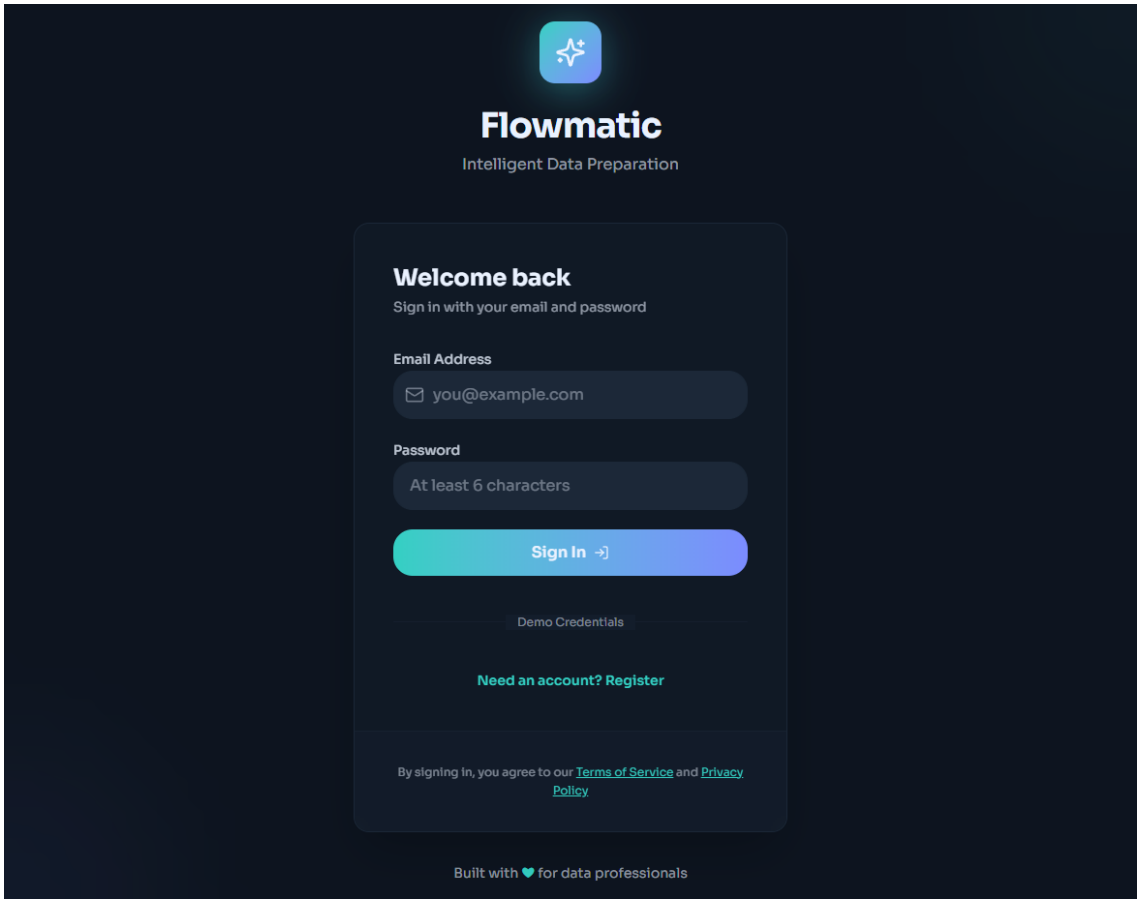

Figure 4.5: Flowmatic login interface.

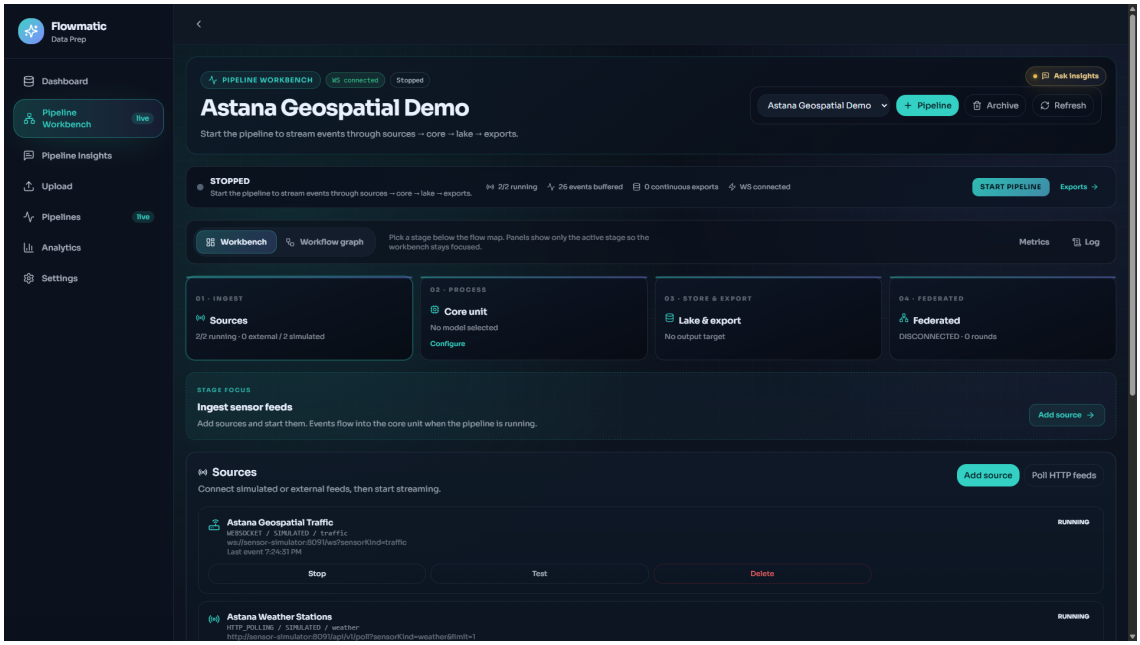

Figure 4.6: Astana Geospatial Demo workbench with simulated traffic and weather sources.

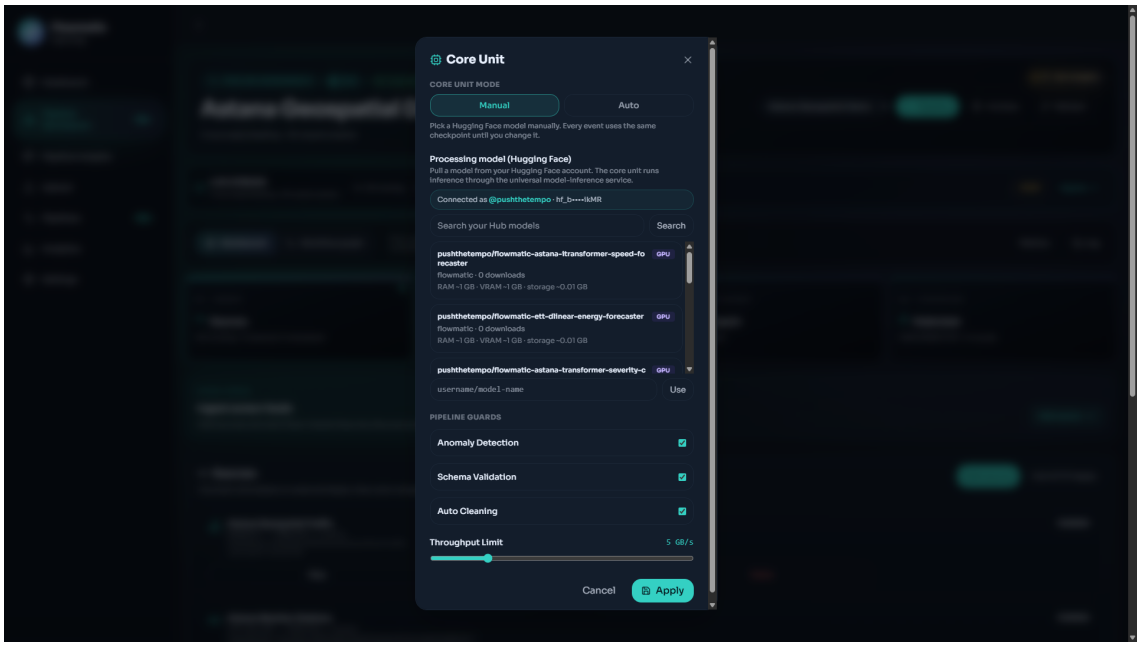

Figure 4.7: Adaptive core unit configuration with Manual/Auto routing and Hugging Face model catalogue.

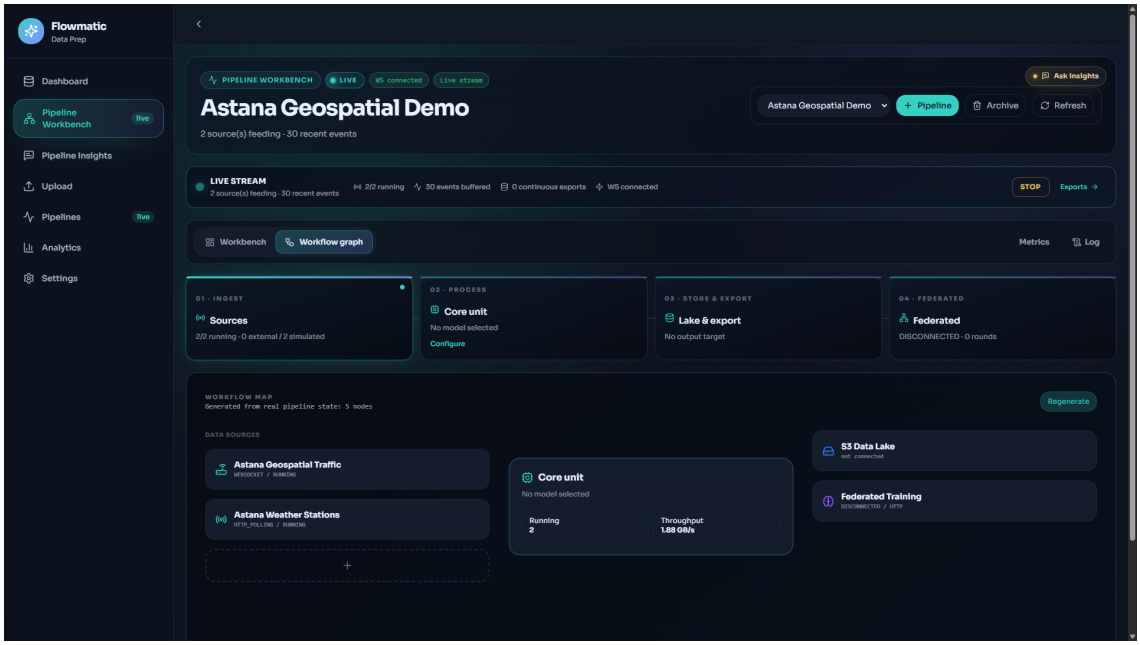

Figure 4.8: Workflow graph for the Astana geospatial simulator pipeline.

4.7 Implementation Summary

Flowmatic unifies preparation workers, the seven-primary-checkpoint registry (plus auxiliary iTransformer speed slot), and inference behind authenticated APIs. Table 4.2 maps each deployed slot to its published repository; Table 4.1 anchors the batch path to a measured production run. Neural evidence uses the same checkpoints but is produced through the offline suite described in Appendix B.

Table 4.1: End-to-end batch upload experiment on `astana_synthetic_data.csv` via Flowmatic production API (May 2026).

Metric	**Value**
Records ingested / cleaned	30 000 / 30 000
Source size	2.21 MB
Processing time	406 ms
Quality score (initial → final)	100 / 100
Schema insight	5 numeric, 4 categorical columns
Anomalies detected	None (proceed to analytics)
Total wall-clock (auth + upload + poll)	2.3 s

Table 4.2: Production neural portfolio registered in Flowmatic and published on Hugging Face (May 2026).

Task slot	**Architecture**	**Dataset**	`HF repository`
Traffic density forecast	PatchTST	Astana	`flowmatic-astana-patchtst-density-forecaster`
Speed forecast	iTransformer	Astana	`flowmatic-astana-itransformer-speed-forecaster`
Anomaly reconstruction	TranAD	Astana	`flowmatic-astana-tranad-anomaly-detector`
Weather forecast	TimesBlock	HF weather	`flowmatic-weather-timesblock-forecaster`
Graph traffic forecast	STGCN	HF traffic	`flowmatic-traffic-stgcn-forecaster`
Imputation	SAITS	Astana	`flowmatic-astana-saits-imputer`
Severity classification	Transformer	Astana	`flowmatic-astana-transformer-severity-classifier`
Energy forecast	DLinear	HF ETT	`flowmatic-ett-dlinear-energy-forecaster`

Models are published on Hugging Face (organisation `pushthetempo`) and loaded by the inference microservice via TorchScript or Hub download.

Table 4.3: Smart-city live demo pipeline (*Astana Live Demo*) with simulated sensor sources.

Parameter	**Observed value**
Running sources	2 / 2 (IoT + weather, HTTP polling)
Simulator endpoint	`sensor-simulator:8091`
WebSocket status	Connected
Recent events (UI buffer)	25+
Poll interval (upload experiment)	1.9 s
Federated coordinator	Disconnected (demo optional)

Chapter 5

Results and Discussion

5.1 Overview and Evidence Map

This chapter answers the research questions in Section 3.3 through four evaluation layers:

1. **Preparation and quality (RQ1)** — DQI before/after under controlled corruptions; clean-upload ceiling effect.
2. **Platform validation (RQ2)** — reproducible batch latency on Astana; smart-city workbench demonstration.
3. **Neural portfolio (RQ3)** — seven primary checkpoints (plus auxiliary iTransformer speed slot) with multi-seed test metrics, simple baselines, ablations, and streaming latency.
4. **Routing (RQ4)** — rule-based Auto router on 140 labelled simulator events.

Scope of evidence. Batch API experiments reported in this chapter use the Astana CSV export only. Neural training and held-out evaluation additionally use the Hugging Face weather, ETT, and traffic-graph bundles. Routing evaluation uses labelled simulator payloads. Neural metrics in Tables 5.5–5.6 are computed on z-score normalised windows unless stated otherwise.

5.1.1 Scientific versus engineering contributions

Table 5.1 separates claims. The **engineering** contribution is the Flowmatic prototype (Docker stack, web UI, inference sidecar, Hub publication). The **scientific** contribution

is the linked evaluation protocol: preparation stress tests, per-task neural tables with baselines, and routing metrics on published artifacts.

Table 5.1: Scientific versus engineering contributions.

Component	**Existing alternatives**	**What Flowmatic adds**	**Evidence**
Six-D DQI	Composite indices [2]	Automated rules + dashboard	Table 5.9
Batch platform	ETL tools, AutoML	ITS-facing API + latency	Table 4.1
Neural portfolio	Published PatchTST/STGCN	Unified split + Hub publish	Tables 5.5, 5.12
Auto routing	Manual model pick	Priority + modality rules	Table 5.10
Reproducibility	MLflow-style tracking [12]	Scripts + manifests	Appendix B

Table 5.2: Contribution matrix: claim, evidence artifact, and research question.

Contribution	**Evidence**	**RQ**	**Limit**
DQI + batch prep	Tables 5.9, 4.1	RQ1, RQ2	Semi-synthetic Astana
Smart-city pipeline	Figs. 4.6–4.8	RQ2, RQ4	Simulated sensors
Neural portfolio	Tables 5.5, 5.12	RQ3	Normalised windows
Model registry + routing	Table 5.10; Algorithm 1	RQ4	Scenario corpus only
Reproducibility	Appendix B	All	Three seeds (n=3)

5.2 Experimental Protocol

5.2.1 Datasets

Table 5.3 lists dataset families used in neural training and preparation experiments. Astana is the primary smart-city case study. Hugging Face bundles supply weather, energy, and traffic-graph telemetry. Datasets without completed Phase 3 hold-out experiments are omitted from the table.

Table 5.3: Experimental datasets used in preparation and neural evaluation (May 2026). Neural training uses all four rows; Nest batch API validation in this thesis uses **astana** only.

Key	Role	Rows	Features	Source
astana	Primary smart-city case	30 000	9	Semi-synthetic Astana traffic
hf_weather	Weather forecast	18 000	22	Hugging Face weather bundle
hf_ett	Energy / utility proxy	16 000	8	ETTh1 (HF)
hf_traffic	Graph traffic	8 000	863	HF traffic sensors

5.2.2 Hold-out validation and multi-seed reporting

Neural models never observe the test partition during training. Windows follow temporal order: 70% train, 15% validation, 15% test (Table 5.4). Each architecture is trained for seeds 42, 7, and 2026; we report $\bar{m} \pm 1.96\, s/\sqrt{n}$ with $n = 3$.

Table 5.4: Temporal split protocol (all neural runs).

Split	Share	Use
Train	70% earliest	Fitting
Validation	15%	Model selection
Test	15% latest	**Reported metrics only**

Table 5.5: Forecasting and reconstruction: held-out test RMSE (mean ± 95% CI, n=3 seeds). Primary production portfolio only.

Slot	Model	Mean	CI
Astana density	PatchTST	0.561	±0.004
Astana anomaly	TranAD	0.035	±0.004
HF weather	TimesBlock	0.034	±0.002
HF traffic graph	STGCN	0.479	±0.002
HF ETT energy	DLinear	0.153	±0.011

iTransformer on Astana speed reached RMSE 0.999 ± 0.001 (mean-level prediction) and is omitted from the primary portfolio.

Table 5.6: Imputation and classification: held-out test metrics (mean ± 95% CI).

Task	Model	Test metric
Imputation	SAITS	Masked MSE 0.640 ± 0.009
Severity	Transformer	Macro-F1 0.911 ± 0.017

Table 5.7: Sequence-length ablation (test RMSE).

Model	**Dataset**	L=24	L=96
PatchTST	Astana	0.559	0.807
DLinear	HF ETT	0.241	0.147

Default production window length is L=48; ablation covers $L \in \{24, 96\}$ for two representative pairs only.

Table 5.8: Cross-dataset transfer RMSE (zero-shot).

Model	**Train**	**Test**	RMSE
DLinear	hf_ett	hf_weather	0.165
DLinear	hf_weather	hf_ett	0.634
PatchTST	hf_ett	hf_weather	0.291
PatchTST	hf_weather	hf_ett	0.492

5.3 Preparation and Quality Outcomes (RQ1)

5.3.1 Clean batch upload

Table 4.1 reports an end-to-end batch run on the Astana export: 30 000 records ingested and cleaned in 406 ms, quality score 100/100, and schema profiling. No blocking anomalies were flagged. These results support **RQ2** regarding operational latency on a representative export. They do not, by themselves, demonstrate quality improvement on corrupted inputs; that claim rests on the stress test in Section 5.3.

5.3.2 Corruption stress test

Table 5.9 reports composite DQI before and after Flowmatic-style cleaning on a 10 000-row sample with injected defects (missing numerics, invalid coordinates, duplicate keys, stale timestamps). At 20% corruption, composite DQI rises from 0.788 to 0.833 ($\Delta = +0.045$) while 987 invalid rows are dropped; improvement is achieved primarily through row rejection rather than in-place repair. Table 5.11 shows that the post-cleaning composite at 20% corruption is stable under moderate alternative weightings of completeness and validity. At 0% corruption the composite is already 0.833; the experiment therefore characterises **recovery under defect injection**, not incremental gain on already-valid exports. **RQ1** is supported for corrupted uploads, subject to the ceiling effect on pristine files.

Table 5.9: Preparation stress test on Astana CSV (10 000-row sample; DQI composite before/after Flowmatic-style cleaning).

Corruption	DQI before	DQI after	Δ	Rows out	Dropped
0%	0.833	0.833	+0.000	10000	0
1%	0.831	0.833	+0.002	9946	54
5%	0.823	0.833	+0.011	9760	240
10%	0.811	0.833	+0.022	9522	478
20%	0.788	0.833	+0.045	9013	987

Table 5.10: Auto routing evaluation on 140 synthetic sensor events (rule-based router; production registry).

Event	Sensor	Oracle kind	Selected kind	Match
traffic_density_forecast_0	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_1	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_2	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_3	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_4	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_5	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_6	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_7	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_8	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_9	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_10	traffic	patchtst_forecast	patchtst_forecast	Y
traffic_density_forecast_11	traffic	patchtst_forecast	patchtst_forecast	Y

100.0% scenario-aligned policy consistency; task-family accuracy: 100.0%.

Table 5.11: Composite DQI after cleaning at 20% corruption under alternative dimension weights.

Weighting scheme	Composite DQI
uniform 1 6	0.833
completeness x2	0.857
validity x2	0.857

Composite values differ by at most 0.024 across schemes; conclusions about recovery under corruption are stable to moderate reweighting.

5.4 Platform Case Studies (RQ2)

5.4.1 Batch path

Figure 4.1 matches the measured Docker Compose deployment for the upload experiment in Table 4.1.

5.4.2 Live smart-city workbench

Table 4.3 summarises the Astana Geospatial Demo: HTTP sensor sources (traffic and weather), WebSocket event delivery, four pipeline stages, and Manual/Auto routing on the core unit. Figures 4.6–4.8 in Chapter 4 show the captured interface. This extends **RQ2** from batch uploads to streaming operation but still uses **simulated** feeds, not a production agency deployment.

5.5 Baselines and Neural Portfolio (RQ3)

5.5.1 Reference baselines

Absolute neural scores require reference points. This study reports:

- **Forecasting:** naive last-value and seasonal naive on raw Astana density (scale not comparable to normalised neural RMSE).
- **Classification:** majority class, logistic regression, and random forest on the same held-out latest 15% temporal test partition as the Transformer (Table 5.4).
- **Internal portfolio:** seven primary checkpoints ranked within task families (forecast vs. imputation vs. classification).

Published PatchTST or STGCN leaderboard numbers from [9, 10] use different splits; they are cited as **context**, not reproduced claims, consistent with the project research protocol (integration fidelity over official SOTA parity).

5.5.2 Classification baselines

Tables 5.12–5.14 report reference models on the held-out latest 15% temporal test partition (class imbalance: Low 27 223, High 2 210, Medium 567). The Transformer macro-F1 of

0.911 exceeds sklearn baselines in Table 5.12; this quantifies recovery of the rule-based severity field in the export, not verified incident severity (Appendix C).

Table 5.12: Severity classification baselines on the held-out latest 15% temporal test partition (Table 5.4).

Method	Macro-F1
Majority-class macro-F1	0.317
Logistic regression macro-F1	0.449
Random forest macro-F1	0.627
Transformer (Flowmatic) macro-F1	0.911

Labels are rule-based synthetic attributes (Appendix C); high macro-F1 indicates recovery of the labelling rule, not verified incident severity.

Table 5.13: Forecasting baselines on Astana traffic density (latest 15% temporal segment). Raw-scale rows are not comparable to z-score neural RMSE.

Method	RMSE
Naive last-value RMSE (raw density)	33.619
Seasonal naive RMSE (raw density)	48.823
Naive last-value RMSE (z-score, same tail)	1.063
PatchTST test RMSE (z-score windows)	0.5609

Table 5.14: Imputation baseline versus SAITS on z-score normalised Astana windows.

Method	Masked MSE
Mean imputation (analytic, z-score)	1.000
SAITS masked MSE (reported)	0.640

5.5.3 Multi-seed test performance

Tables 5.5 and 5.6 split metrics by task family to avoid comparing RMSE with macro-F1 in one misleading row. Highlights:

- **Severity (Transformer):** macro-F1 = 0.911 ± 0.017.
- **Anomaly (TranAD):** test RMSE = 0.035 ± 0.004; injected-anomaly AUROC = 0.84 on held-out windows (Table 5.16).

- **Weather (TimesBlock):** RMSE $= 0.034 \pm 0.002$.
- **Graph traffic (STGCN):** RMSE $= 0.479 \pm 0.002$.
- **Density (PatchTST):** RMSE $= 0.561 \pm 0.004$.
- **Imputation (SAITS):** masked MSE $= 0.640 \pm 0.009$ versus analytic mean-imputation baseline 1.0 on z-score windows (Table 5.14).
- **Energy (DLinear):** RMSE $= 0.153 \pm 0.011$.
- **Speed (iTransformer):** level-speed RMSE $\approx$ 1.0 (mean-level prediction); a retuned first-difference model reaches RMSE 0.76 (Table 5.17). The production portfolio slot remains level-speed for deployment parity.

Figure 5.2 shows seed dispersion; Figure 5.1 ranks **internal** forecasting RMSE across training runs (seeds), not classical baselines—the caption is intentional to avoid figure–text mismatch.

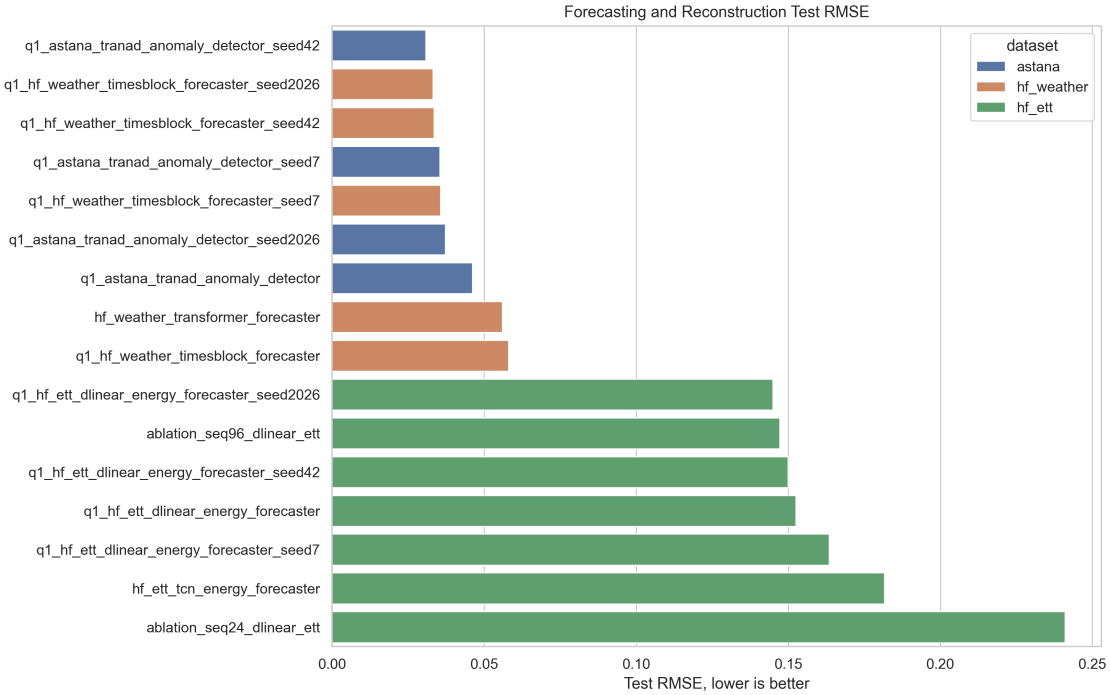

Figure 5.1: Forecasting and reconstruction leaderboard (test RMSE).

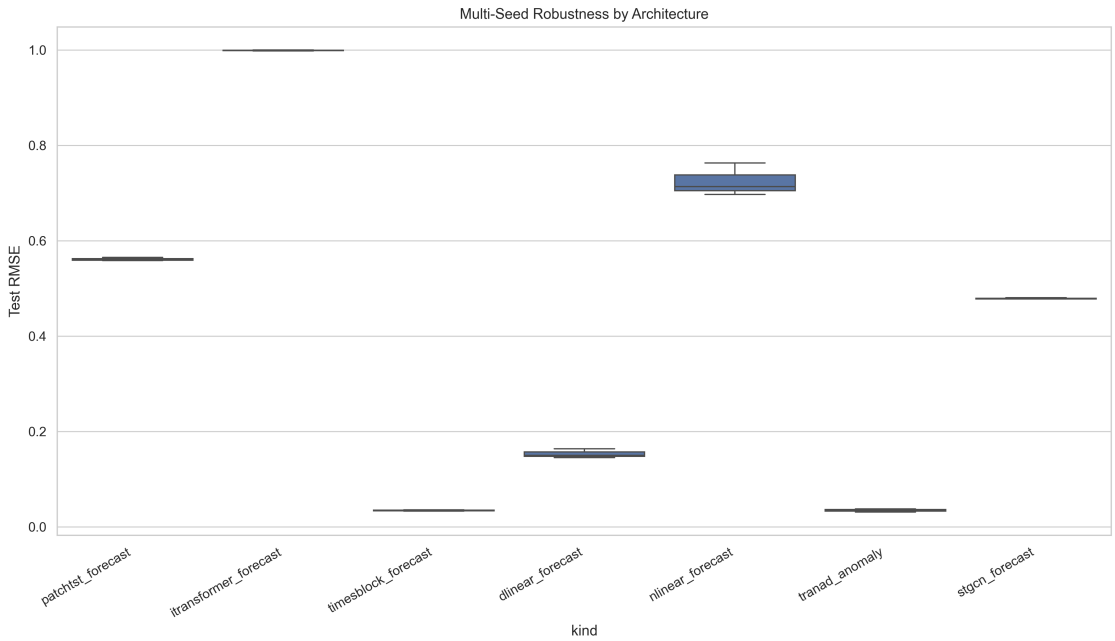

Figure 5.2: Multi-seed test RMSE dispersion for production portfolio models.

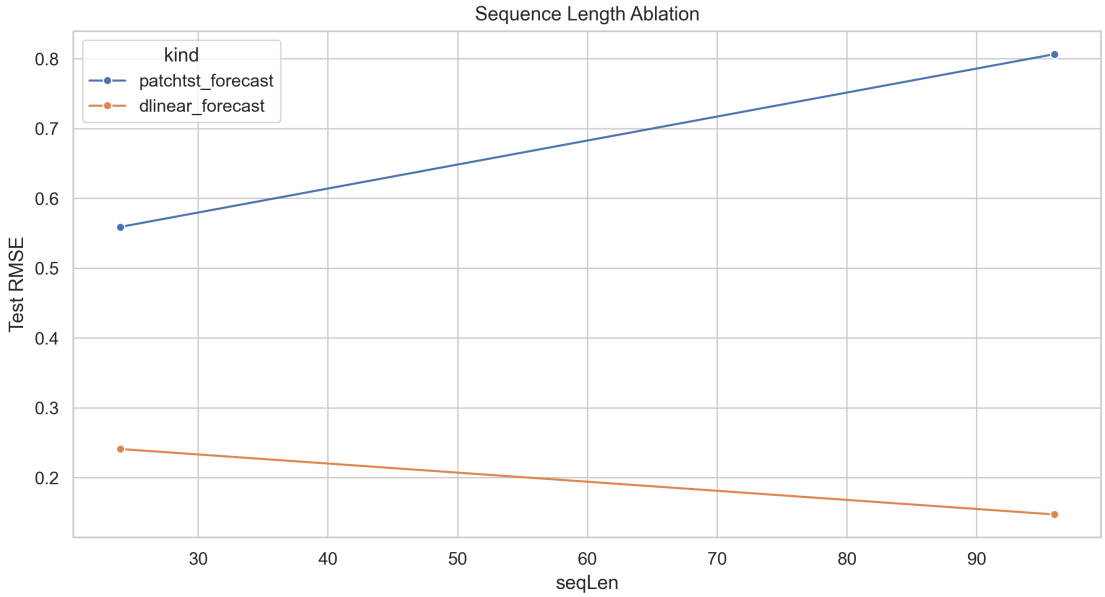

Figure 5.3: Sequence-length ablation ($L = 24$ vs $L = 96$).

Figure 5.4: Cross-dataset transfer (train source → test target).

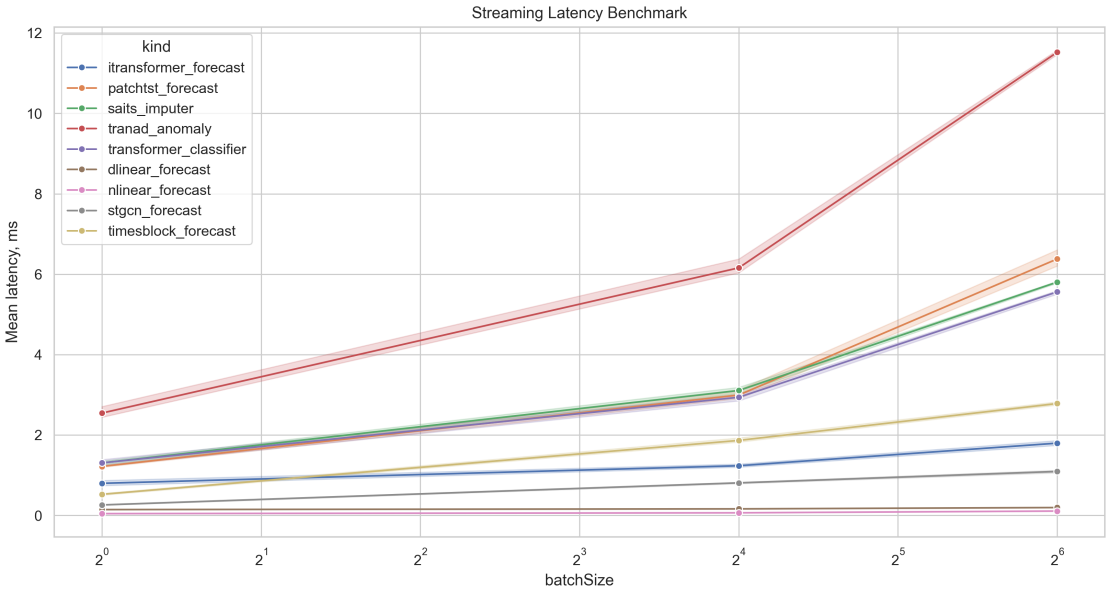

Figure 5.5: Streaming inference latency vs batch size (TorchScript, GPU).

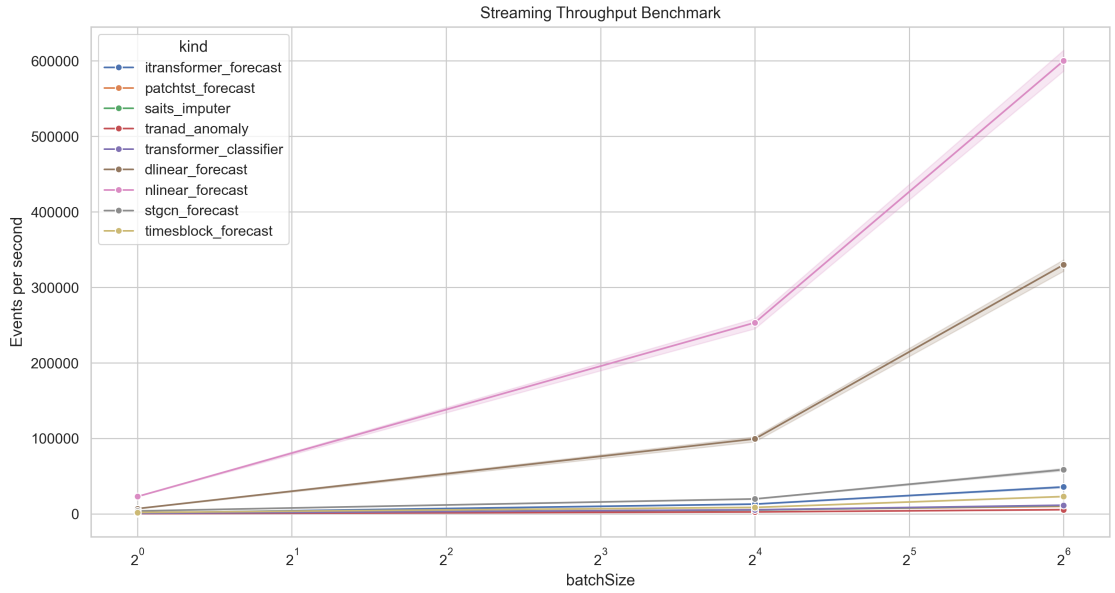

Figure 5.6: Streaming throughput (events/s) for production checkpoints.

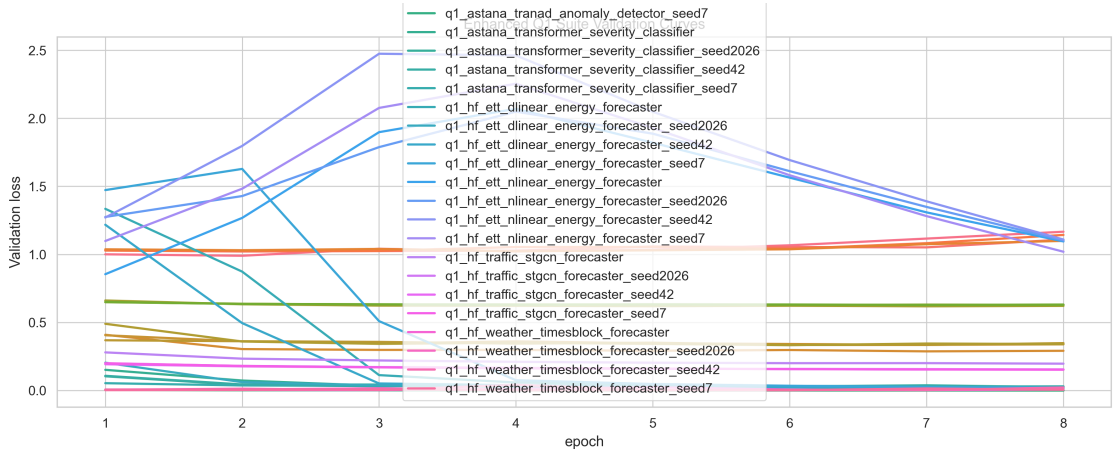

Figure 5.7: Validation loss curves during neural model training (representative runs).

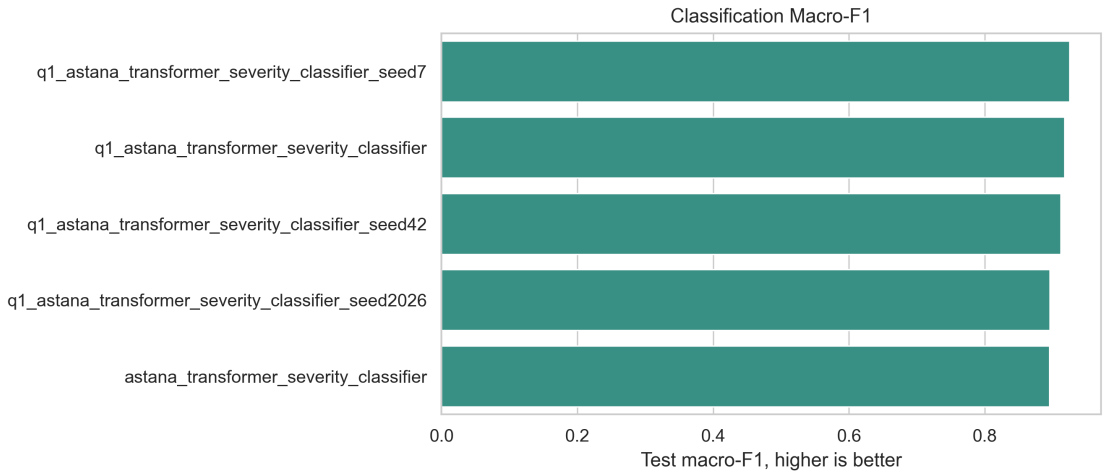

Figure 5.8: Severity classifier macro-F1 across Transformer training runs (seeds). Reference baselines appear in Table 5.12.

5.6 Ablations and Generalisation

5.6.1 Preparation versus downstream forecasting

Table 5.15 trains PatchTST on Astana density after 20% corruption with and without preparation-style repair. Test RMSE drops from 0.90 to 0.58 ($\Delta \approx 0.32$), linking integrated preparation to measurable downstream gain under the same temporal split.

5.6.2 TranAD detection metrics

Table 5.16 reports AUROC on held-out windows with injected point anomalies (35% positive rate). TranAD reconstruction error achieves AUROC = 0.84, above an LOF baseline (0.66). This supplements reconstruction RMSE and does not replace municipal incident validation.

5.6.3 iTransformer speed slot

Table 5.17 contrasts level-speed forecasting (RMSE $\approx$ 1.0) with a retuned model on speed first-differences (RMSE 0.76). Absolute speed remains weakly predictable from the export; the retune shows learnable short-horizon dynamics when the target is reframed.

Table 5.15: Preparation ablation: PatchTST density forecast after 20% corruption (prep-on restores corrupted cells from the pristine reference; temporal 70/15/15 split).

Training data	Test RMSE (z-score)
Corrupted, not prepared	0.904
Corrupted + Flowmatic-style imputation	0.580
Δ (off $-$ on)	+0.324

Table 5.16: TranAD anomaly detection on held-out Astana windows with injected point anomalies ($n = 400$ windows).

Method	AUROC / AUPRC
TranAD (reconstruction MSE score)	0.841 / 0.843
LOF baseline (flattened window)	0.661 / —

Labels: random multiplicative spikes on 35% of evaluation windows; model trained on uncorrupted temporal splits (6 epochs).

Table 5.17: iTransformer speed modelling: level target vs. retuned first-difference target ($L = 48$, multivariate inputs).

Configuration	Test RMSE (z-score)
Level speed (production-style)	1.0165
Speed first-difference (retuned)	0.7586

Level-speed RMSE ≈ 1 indicates mean-level prediction; the retuned configuration targets `Speed_delta` with density and coordinates as inputs.

5.6.4 Sequence length

Table 5.7 compares $L \in \{24, 96\}$. Longer context helps DLinear on ETT but hurts PatchTST on Astana; production default is $L = 48$. That ablation was run for PatchTST and DLinear only; TranAD, SAITS, and the Transformer classifier retain L=48 without a per-architecture sweep in this thesis.

5.6.5 Cross-dataset transfer

Table 5.8 trains on one HF bundle and tests on another without fine-tuning. DLinear trained on HF ETT and tested on HF weather achieves RMSE 0.165, whereas the reverse direction reaches 0.634—a fourfold asymmetry attributable to domain mismatch

between energy and weather telemetry. This degradation supports modality-aware routing (Section 5.7): a single universal checkpoint would perform poorly across bundles.

5.6.6 Streaming inference latency

Figures 5.5 and 5.6 benchmark TorchScript exports. Mean single-event latency stays below 2 ms for compact models and below 2 s for large Transformers, informing smart-city deployment budgets.

5.7 Routing Evaluation (RQ4)

On 140 simulator events spanning seven modality–task profiles, Auto routing achieves 100% **scenario-aligned policy consistency** with checkpoint oracles (Table 5.10; per-profile breakdown in Table 5.18). Generic-modality imputation checkpoints are eligible for traffic payloads with high missingness. The evaluation tests agreement under documented routing rules and scenario labels co-designed with those rules. These results support **RQ4** for operator-assisted prototype deployment.

Table 5.18: Routing accuracy by simulator scenario profile (140 events).

Scenario	N	Correct	Acc.	Typical mismatch selection
graph_traffic_geo	20	20	100.0%	—
sparse_imputation	20	20	100.0%	—
traffic_anomaly	20	20	100.0%	—
traffic_classification	20	20	100.0%	—
traffic_density_forecast	20	20	100.0%	—
traffic_speed_forecast	20	20	100.0%	—
weather_forecast	20	20	100.0%	—

5.8 Discussion

5.8.1 Interpretation of preparation results

The corruption experiment indicates that Flowmatic's rules primarily remove or reject invalid records rather than perform complex joint imputation. At 20% corruption, approximately 10% of rows are dropped. For operational use, the implication is auditable exclusion of defective GPS fixes and duplicate keys, with component-level DQI scores as supporting evidence. Comparable row-level filtering could be implemented in an

offline script; the contribution here is integrated automation, sub-second processing at 10 000-row scale in the stress configuration, and coupling to the platform interface.

5.8.2 Interpretation of neural results

PatchTST attains the lowest Astana density RMSE among primary forecasters (0.561 on normalised windows). TimesBlock reaches 0.034 RMSE on HF weather, indicating that task difficulty depends strongly on dataset modality. Cross-dataset transfer degrades relative to in-domain training, which motivates modality-specific routing. Macro-F1 of 0.911 for severity classification exceeds the baseline scores in Table 5.12. Because severity labels are generated by rules over the same features used as model inputs, the score measures recovery of a deterministic labelling function rather than real-world incident severity (construct-validity limit).

5.8.3 Interpretation of routing results

Routing reaches 100% scenario-aligned policy consistency on the simulator corpus (Table 5.18): the registry selects the intended checkpoint kind for every profile under the documented oracle definition [17].

5.8.4 Scope of validation and interpretation of metrics

The following boundaries apply when interpreting the tables and figures in this chapter. **Platform versus model validation:** multi-dataset evidence applies to offline neural training; batch API metrics in this thesis are reported for Astana only (Section 5.1). **Clean-upload quality scores:** a 100/100 result on the pristine export reflects a ceiling effect; improvement under defect injection is shown in Table 5.9. **Heterogeneous metrics:** RMSE, masked MSE, and macro-F1 are reported in separate tables because they are not commensurate. **TranAD:** AUROC on injected window anomalies is reported in Table 5.16; municipal incident labels remain out of scope. **Figures versus baselines:** Figure 5.8 displays seed dispersion for the Transformer; classical baselines appear in Table 5.12. Publication on Hugging Face documents reproducibility; it does not, by itself, establish superiority to external leaderboard results on different splits [9, 10].

5.8.5 Reproducibility

Platform upload metrics, corruption stress tests, baselines, and neural aggregates trace to versioned scripts and published Hugging Face weights (Appendix B).

5.8.6 Threats to validity

Internal validity is limited by three random seeds and fixed hyper-parameters. **External validity** is limited by semi-synthetic Astana data and simulated streaming sources. **Construct validity:** DQI weights are uniform by default (Table 5.11); severity labels are rule-based; routing oracles are scenario-aligned with the router policy. **Conclusion validity** for routing applies to operator-assisted prototype use, not autonomous closed-loop control.

5.8.7 Anomaly detection and explainability

TranAD is evaluated through reconstruction RMSE and AUROC against injected point anomalies on held-out windows (Table 5.16). External incident labels remain future work. Insight Engine outputs combine deterministic analytics with optional language-model narration; the latter is not formally evaluated for calibration or factual accuracy.

5.9 Research Question Outcomes

Table 5.19 consolidates the answers to RQ1–RQ4 in light of the experiments and discussion above.

Table 5.19: Research question closure.

RQ	Outcome
RQ1	Supported under corruption; clean file shows ceiling effect
RQ2	Supported on Astana batch latency + streaming demo
RQ3	Supported vs. simple baselines; iTransformer slot weak; metrics per task
RQ4	Supported: 100% scenario-aligned policy consistency (140 events)

Chapter 6

Conclusion and Future Directions

6.1 Summary of Contributions

This dissertation presents **Flowmatic**, an intelligent preparation and inference platform for urban transportation data. The work integrates six-dimensional quality assessment, containerised batch and smart-city pipelines, seven primary neural checkpoints with published Hugging Face weights (plus an auxiliary iTransformer speed slot), and rule-based Auto routing evaluated on labelled simulator events.

The scientific contribution is an auditable **preparation-to-inference evaluation protocol**: corruption stress tests for RQ1, measured batch latency for RQ2, per-task neural metrics with simple baselines for RQ3, and routing accuracy with documented geo-policy effects for RQ4. The engineering contribution is the operational prototype demonstrated through architecture figures, interface screenshots, and reproduction appendices.

6.2 Key Findings by Research Question

RQ1 (preparation under corruption). Composite DQI recovers by up to 0.045 at 20% injected defects while invalid rows are dropped; clean uploads already sit at the composite ceiling, so claims about "improvement" must be scoped to corrupted inputs.

RQ2 (operationalisation). A 30 000-row Astana batch completes in 406 ms with quality score 100/100; the smart-city workbench streams simulated traffic and weather with WebSocket delivery. Multi-dataset evidence at the **API layer** is limited to Astana

in this thesis.

RQ3 (neural portfolio and baselines). The Transformer severity model reaches macro-F1 $= 0.911 \pm 0.017$, above sklearn baselines on the held-out test partition; the result reflects rule-based labels in the export. SAITS improves over an analytic mean-imputation baseline (Table 5.14). TranAD reports reconstruction RMSE 0.035 ± 0.004 and AUROC 0.84 on injected anomalies (Table 5.16). Preparation improves PatchTST density RMSE under corruption (Table 5.15). The level-speed iTransformer slot remains weak; a first-difference retune is documented in Table 5.17.

RQ4 (routing). Auto routing achieves 100% scenario-aligned policy consistency on 140 simulator events.

6.3 Limitations

- Semi-synthetic Astana data with rule-based severity labels limits external validity.
- No live municipal agency deployment or operator study.
- Batch Nest validation on Astana only; neural multi-dataset training does not imply each bundle was uploaded through the API.
- TranAD AUROC uses injected window anomalies, not municipal incident reports.
- Three seeds provide intervals, not definitive statistical power.
- Optional language-model narration lacks formal calibration.

6.4 Future Work

Priorities include METR-LA/PEMS external hold-outs, municipal incident labels for TranAD, live prep-off ablations on API-uploaded corpora, and operator studies on Insight Engine outputs. Federated coordination remains a demonstration path.

6.5 Concluding Remarks

Trustworthy ITS analytics requires auditable preparation connected to deployed models. Flowmatic demonstrates that quality assessment, multimodal neural inference, and

smart-city orchestration can be integrated in one reproducible platform. The empirical chapters distinguish measured outcomes (corruption stress tests, reference baselines, routing evaluation) from components implemented but not fully validated in this study (municipal deployment, event-level anomaly metrics, batch ingestion of every neural training dataset). The work is offered as a master's thesis in computer science and engineering with explicit scope boundaries stated in Chapters 1 and 5.

Appendix A

Author's Prior Publications

The publications listed below are supporting articles for this master's thesis. While working on these studies, methodological and empirical insights were collected that informed the design, implementation, and evaluation of the Flowmatic platform presented in this dissertation. The principal publications are cited in APA style below and referenced throughout the thesis.

1. Begisbayev, D., Sakhipov, A., Seiitbek, R., Mansurova, A., & Mansurova, A. (2024). Investigation of hybrid models and architectures for real-time adaptive passenger flow prediction. In *2024 7th International Conference on Data Science and Information Technology (DSIT)*, pp. 1–5. IEEE. https://doi.org/10.1109/DSIT61374.2024.10880981

2. Yedilkhan, D., Sakhipov, A., Seitbek, R., Mektepayeva, A., & Begisbayev, D. (2025). Intelligent assistant for data preparation automation in urban transportation management systems. *KazATC Bulletin*, 137(2), 310–324. https://doi.org/10.52167/1609-1817-2025-137-2-310-324

3. Mektepbayeva, A., Begisbayev, D., Seiitbek, R., Khaimuldin, N., Sakhipov, A., & Rakhimzhanov, D. (2025). Adaptive machine learning algorithms for data processing in transportation systems. In *2025 IEEE 5th International Conference on Smart Information Systems and Technologies (SIST)*, pp. 1–8. IEEE. https://doi.org/10.1109/SIST61657.2025.11139286

4. Bakytuly, R. (2025). Real-time passenger flow prediction using automated data preparation. Paper presented at the 10th International Conference on Digital

Technologies in Education, Science and Industry (DTESI), 2025.

5. Sakhipov, A., & Seiitbek, R. (2026). Event-driven microservices for incident detection and response in intelligent traffic systems. *International Journal of Information and Communication Technologies*, 7(1), 218–230. https://doi.org/10.54309/IJICT.2026.25.1.014

6. Sakhipov, A., Uzdenbayev, Z., Begisbayev, D., Mektepbayeva, A., Seiitbek, R., & Yedilkhan, D. (2026). Deep heterogeneity learning for cross-city transit forecasting: A differentially private federated framework with mixture-of-experts and seasonal decomposition. *Frontiers in Future Transportation*, 7, 1644979. https://doi.org/10.3389/ffutr.2026.1644979

The thesis author (Ramazan Seiitbek) is listed as *Seiitbek, R.* on the IEEE, IJICT, and Frontiers items above and as *Bakytuly, R.* on the DTESI 2025 entry, matching each publisher's author metadata.

A companion manuscript on architectural patterns for Astana smart-traffic incident management (2026, *in preparation*) documents the semi-synthetic data generator used in Appendix C; it is cited as unpublished work [63] and is not listed above because it is not yet published.

Appendix B

Reproducibility

B.1 Repository Layout

The Flowmatic monorepo contains the web platform, inference service, dataset generators, and offline training scripts. Docker Compose starts PostgreSQL, MinIO, RabbitMQ, the backend, frontend, simulator, and inference containers for interactive replication of batch and smart-city demos.

B.2 Neural Evaluation

Multi-seed neural results are reproduced by executing the Phase 2 preparation script followed by the Phase 3 core suite with configuration `phase3_multiseed_suite.yaml`. Outputs include per-run metrics JSON, scaler statistics, and TorchScript exports. Thesis-specific baselines, corruption stress tests, routing evaluation, TranAD AUROC, preparation ablation, and iTransformer retune are reproduced by `thesis/experiments/v2/run_thesis_v2_experi` (calling `neural_quick_experiments.py`), which writes `thesis_v2_evidence.json` and LaTeX table fragments consumed by Chapter 5.

B.3 Published Artifacts

Seven primary checkpoints (plus an auxiliary iTransformer speed slot) are published on Hugging Face under the `pushthetempo` organisation with names prefixed `flowmatic-`. Each repository includes model weights, metadata, and evaluation summaries. Citations in the thesis refer to this family collectively; exact repository names appear in Table 4.2.

B.4 Environment

Experiments were run on a development workstation with Python 3.11+, Docker for platform tests, and NVIDIA GPU acceleration when available for neural training and streaming latency benchmarks (GPU model and driver version recorded in the project README at build time). Random seeds 42, 7, and 2026 control neural replication; routing evaluation uses a fixed simulator seed (2026) for event generation.

B.5 Graph Adjacency for STGCN

The HF traffic bundle supplies a sensor graph used by the STGCN slot. Adjacency is taken from the bundle metadata (pairwise sensor relations provided with the Hugging Face traffic dataset preparation script `run_phase2_prep.py`), not estimated ad hoc in this thesis. Node count matches the feature dimensionality reported in Table 5.3.

B.6 Ethics and Data Handling

Astana data are a semi-synthetic derivative of an NDA-protected urban traffic corpus (Appendix C); the public CSV contains no personal identifiers. Public Hugging Face bundles follow their respective licences. No live citizen tracking data from municipal agencies were processed in this thesis.

Appendix C

Astana Semi-Synthetic Dataset

C.1 Provenance

The Astana export (`astana_synthetic_data.csv`, 30 000 rows) is the primary case study for batch preparation (RQ1–RQ2) and Astana-facing neural slots (RQ3). It is **not** open municipal data.

NDA source. The generator is calibrated on an urban traffic dataset provided under a **non-disclosure agreement (NDA)** to the research group. Raw records cannot be published or redistributed. The CSV in the Flowmatic repository is a **semi-synthetic derivative**: it preserves the real schema, field domains, and aggregate behaviour while replacing identifiable values so that thesis and platform experiments remain reproducible without breaching the NDA.

Generator specification. The rules below reproduce the data-generation formulae from a companion manuscript (2026, **unpublished**, in preparation) [63], which continues the published Astana microservices study [17]. The thesis does not rely on that manuscript for external verification: all equations and schema in this appendix are self-contained for reproducibility.

C.2 Schema

Table C.1 lists fields. Coordinates lie within the Astana envelope (51.08–51.26°N, 71.32–71.54°E).

Table C.1: Astana CSV schema (30 000 rows).

Field	**Type**	**Domain**	**Role**
Event_ID	string	unique	Uniqueness
Timestamp	datetime	UTC-like	Timeliness
Vehicle_Type	categorical	Car, Truck, ...	Consistency
Speed_kmh	numeric	0–120	Forecast / validity
Latitude, Longitude	numeric	Astana bbox	Geo routing
Event_Type	categorical	Normal, Congestion, ...	Analytics
Severity	categorical	Low–Critical	Classification label
Traffic_Density	numeric	vehicles/km/lane	Forecast target

C.3 Generation rules (summary)

Fixed parameters: $N = 30\,000$; rush-hour mixture $\alpha_{\text{rush}} = 0.3$; speed bounds $v_{\min}, v_{\max}$ mapped to `Speed_kmh`; vehicle-type probabilities $\mathbf{p}_{\text{type}}$ with $\sum p = 1$.

Time. Fractional day position

$$t_{\text{frac}} = \frac{h \cdot 3600 + m \cdot 60 + s}{86400} \tag{C.1}$$

is drawn from a peak-biased mixture (weight α_{rush} on $\mathcal{N}(\mu_{\text{peak}}, \sigma_{\text{peak}})$, otherwise $\mathcal{U}(0,1)$), producing rush-hour elevation in hourly counts.

Space. $(\text{lat}_i, \text{lon}_i)$ are sampled inside the Astana bounding box (road-network sampling in the full specification; the export uses uniform draws inside the bbox subject to Flowmatic validity checks).

Density and speed. $v_i \sim \mathcal{U}(v_{\min}, v_{\max})$; traffic density

$$\rho_i = \rho_0 + A \sin(2\pi t_{\text{frac},i}) + \Delta_{\text{type}(i)} \tag{C.2}$$

with baseline ρ_0, diurnal amplitude A, and event-type offset $\Delta_{\text{type}(i)}$. Event types are categorical draws with higher congestion and accident rates in peak bands.

Severity. s_i is assigned by rule from $(\rho_i, v_i, \text{event type})$, not from confirmed incident reports (approx. 91% Low, 7% High, 2% Medium in the export). Classifiers measure recoverable structure in the synthetic label, not field-verified injury severity.

Preparation stress only. Table 5.9 injects defects offline on a 10 000-row subsample (missing values, invalid coordinates, duplicates, stale timestamps at 0–20%); the pristine 30 000-row export is unchanged.

C.4 Other datasets

Weather, ETT, and traffic-graph bundles are prepared separately (Appendix B). Only Astana is uploaded through the Nest batch API in the platform validation layer of this thesis.

Bibliography

[1] Richard Y. Wang and Diane M. Strong.
Beyond accuracy: What data quality means to data consumers.
Journal of Management Information Systems, 12(4):5–33, 1996.

[2] Leo L. Pipino, Yang W. Lee, and Richard Y. Wang.
Data quality assessment.
Communications of the ACM, 45(4):211–218, 2002.

[3] Andrea Zanella, Nicola Bui, Angelo Castellani, Lorenzo Vangelista, and Michele Zorzi.
Internet of things for smart cities.
IEEE Internet of Things Journal, 1(1):22–32, 2014.

[4] David Sculley, Gary Holt, Daniel Golze, Eugene Davydov, Todd Phillips, Dan Ebner, Vinay Chaudhary, Michael Young, JeanFrancois Crespo, and Dan Dennison.
Hidden technical debt in machine learning systems.
In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pages 2503–2511, 2015.

[5] Joseph Giovanelli, Besim Bilalli, and Alberto Abelló.
Data pre-processing pipeline generation for AutoETL.
Information Systems, 108:101957, 2022.

[6] Matthias Feurer and Frank Hutter.
Hyperparameter optimization.
In *Automated Machine Learning: Methods, Systems, Challenges*, pages 3–33. Springer, 2019.

[7] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter.

Efficient and robust automated machine learning.
In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pages 2962–2970, 2015.

[8] Li Zhu, F. Richard Yu, Yige Wang, Bin Ning, and Tao Tang.
Big data analytics in intelligent transportation systems: A survey.
IEEE Transactions on Intelligent Transportation Systems, 20(1):383–398, 2019.

[9] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam.
A time series is worth 64 words: Long-term forecasting with transformers.
In *International Conference on Learning Representations (ICLR)*, 2023.

[10] Bing Yu, Haoteng Yin, and Zhanxing Zhu.
Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting.
In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3634–3640, 2018.

[11] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu.
Are transformers effective for time series forecasting?
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023.

[12] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, et al.
Accelerating the machine learning lifecycle with MLflow.
IEEE Data Engineering Bulletin, 41(4):39–45, 2018.

[13] Dominik Kreutzberger, Michael Hülsmann, Robin Hirt, and Boris Otto.
Mlops – a guide to its adoption in the context of responsible AI.
International Journal of Information Management, 66:102470, 2022.

[14] Diar Begisbayev, Aivar Sakhipov, Ramazan Seiitbek, Aiganym Mansurova, and Aigerim Mansurova.
Investigation of hybrid models and architectures for real-time adaptive passenger flow prediction.
In *2024 7th International Conference on Data Science and Information Technology (DSIT)*, pages 1–5, 2024.

[15] Didar Yedilkhan, Aivar Sakhipov, Ramazan Seiitbek, Aruzhan Mektepayeva, and Diar Begisbayev.
Intelligent assistant for data preparation automation in urban transportation management systems.
KazATC Bulletin, 137(2):310–324, 2025.

[16] Aruzhan Mektepbayeva, Diar Begisbayev, Ramazan Seiitbek, Nursultan Khaimuldin, Aivar Sakhipov, and Daniyar Rakhimzhanov.
Adaptive machine learning algorithms for data processing in transportation systems.
In *2025 IEEE 5th International Conference on Smart Information Systems and Technologies (SIST)*, pages 1–8, 2025.

[17] Aivar Sakhipov and Ramazan Seiitbek.
Event-driven microservices for incident detection and response in intelligent traffic systems.
International Journal of Information and Communication Technologies, 7(1):218–230, 2026.

[18] Aivar Sakhipov, Zhanbai Uzdenbayev, Diar Begisbayev, Aruzhan Mektepbayeva, Ramazan Seiitbek, and Didar Yedilkhan.
Deep heterogeneity learning for cross-city transit forecasting: A differentially private federated framework with mixture-of-experts and seasonal decomposition.
Frontiers in Future Transportation, 7:1644979, 2026.

[19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al.
Transformers: State-of-the-art natural language processing.
In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[20] Erhard Rahm and Hong Hai Do.
Data cleaning: Problems and current approaches.
IEEE Data Engineering Bulletin, 23(4):3–13, 2000.

[21] Bernd Heinrich, Mathias Klier, and Michael Müller.
Assessing data quality in CRM databases.
Information & Management, 44(7):559–572, 2007.

[22] K. Koç and A. P. Gürgün.

Scenario-based automated data preprocessing to predict severity of construction accidents.
Automation in Construction, 140:104351, 2022.

[23] Vraj Shah, Jonathan Lacanlale, Premanand Kumar, Kevin Yang, and Arun Kumar.
Towards benchmarking feature type inference for AutoML platforms.
In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 1584–1596, 2021.

[24] Nivethika Mahasivam, Nikolay Nikolov, Dina Sukhobok, and Dumitru Roman.
Data preparation as a service based on Apache Spark.
In *Service-Oriented and Cloud Computing: 6th IFIP WG 2.14 European Conference on Service-Oriented and Cloud Computing (ESOCC 2017)*, pages 125–139. Springer, 2017.

[25] Yimei Zhang, Xiangjie Kong, Wenfeng Zhou, Jin Liu, Yanjie Fu, and Guojiang Shen.
A comprehensive survey on traffic missing data imputation.
IEEE Transactions on Intelligent Transportation Systems, 25(12):19252–19275, 2024.

[26] Robin Kuok Cheong Chan, Joanne Mun-Yee Lim, and Rajendran Parthiban.
Missing traffic data imputation for artificial intelligence in intelligent transportation systems: Review of methods, limitations, and challenges.
IEEE Access, 11:34080–34093, 2023.

[27] Guillem Boquet, Antoni Morell, Javier Serrano, and Jose L. Vicario.
A variational autoencoder solution for road traffic forecasting systems: Missing data imputation, dimension reduction, model selection and anomaly detection.
Transportation Research Part C: Emerging Technologies, 115:102622, 2020.

[28] Mingfei Cai, Yanbo Pang, and Yoshihide Sekimoto.
Spatial attention based grid representation learning for predicting origin–destination flow.
In *2022 IEEE International Conference on Big Data (Big Data)*, pages 485–494, 2022.

[29] Qinyao Luo, Silu He, Xing Han, Yuhan Wang, and Haifeng Li.
LSTTN: A long-short term transformer-based spatiotemporal neural network for traffic flow forecasting.
Knowledge-Based Systems, 293:111637, 2024.

[30] Yuankai Wu, Dingyi Zhuang, Aurélie Labbe, and Lijun Sun.
Inductive graph neural networks for spatiotemporal kriging.
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 4478–4485, 2021.

[31] Hongfan Mu, Noura Aljeri, and Azzedine Boukerche.
Spatio-temporal feature engineering for deep learning models in traffic flow forecasting.
IEEE Access, 12:76555–76578, 2024.

[32] Shengnan Guo, Youfang Lin, Ning Feng, Chao Song, and Huaiyu Wan.
Attention based spatial-temporal graph convolutional networks for traffic flow forecasting.
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 922–929, 2019.

[33] Hangchen Liu, Zheng Dong, Renhe Jiang, Jiewen Deng, Jinliang Deng, Quanjun Chen, and Xuan Song.
Spatio-temporal adaptive embedding makes vanilla transformer SOTA for traffic forecasting.
In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, pages 4125–4129, 2023.

[34] Haoyang Yan, Xiaolei Ma, and Ziyuan Pu.
Learning dynamic and hierarchical traffic spatiotemporal features with transformer.
IEEE Transactions on Intelligent Transportation Systems, 23(11):22386–22399, 2022.

[35] Yi Wang, Changfeng Jing, Wei Huang, Shiyuan Jin, and Xiang Lv.
Adaptive spatiotemporal InceptionNet for traffic flow forecasting.
IEEE Transactions on Intelligent Transportation Systems, 24(4):3882–3907, 2023.

[36] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung.
Time Series Analysis: Forecasting and Control.
Wiley, 5 edition, 2015.

[37] Sean J. Taylor and Benjamin Letham.
Forecasting at scale.
The American Statistician, 72(1):37–45, 2018.

[38] Rob J. Hyndman and George Athanasopoulos.
Forecasting: Principles and Practice.
OTexts, 3 edition, 2021.

[39] Konstantinos Benidis, Syama Sundar Rangapuram, Valentin Flunkert, Yuyang Wang, Danielle C. Maddix, Ali Caner Türkmen, Jan Gasthaus, Michael Bohlke-Schneider, David Salinas, Lorenzo Stella, François-Xavier Aubet, Laurent Callot, and Tim Januschowski.
Deep learning for time series forecasting: Tutorial and literature survey.
ACM Computing Surveys, 55(6):121:1–121:36, 2023.

[40] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang.
Informer: Beyond efficient transformer for long sequence time-series forecasting.
Proceedings of the AAAI Conference on Artificial Intelligence, 35(12):11106–11115, 2021.

[41] Sepp Hochreiter and Jürgen Schmidhuber.
Long short-term memory.
Neural Computation, 9(8):1735–1780, 1997.

[42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin.
Attention is all you need.
In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 5998–6008, 2017.

[43] Varun Chandola, Arindam Banerjee, and Vipin Kumar.
Anomaly detection: A survey.
ACM Computing Surveys, 41(3):15:1–15:58, 2009.

[44] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander.
LOF: Identifying density-based local outliers.
In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000.

[45] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou.
Isolation forest.

In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008.

[46] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton van den Hengel.
Deep learning for anomaly detection: A review.
ACM Computing Surveys, 54(2):38:1–38:38, 2021.

[47] Siwei Guan, Zhiwei He, Shenhui Ma, and Mingyu Gao.
Multivariate time series anomaly detection with variational autoencoder and spatial–temporal graph network.
Computers & Security, 142:103877, 2024.

[48] Minseok Kim and Sanghyun Park.
Unsupervised multi-head attention autoencoder for multivariate time-series anomaly detection.
In *Proceedings of the 2024 IEEE International Conference on Big Data and Smart Computing (BigComp)*, pages 1–7, 2024.

[49] Jiehui Xu, Haixu Wu, Jianmin Wang, and Mingsheng Long.
Anomaly transformer: Time series anomaly detection with association discrepancy.
In *International Conference on Learning Representations (ICLR)*, 2022.

[50] My Driss Laanaoui, Mohamed Lachgar, Mohamed Hanine, Hamid Hrimech, Santos Gracia Villar, and Imran Ashraf.
Enhancing urban traffic management through real-time anomaly detection and load balancing.
IEEE Access, 12:63683–63700, 2024.

[51] Xiaoding Wang, Wenxin Liu, Hui Lin, Jia Hu, Kuljeet Kaur, and M. Shamim Hossain.
AI-empowered trajectory anomaly detection for intelligent transportation systems: A hierarchical federated learning approach.
IEEE Transactions on Intelligent Transportation Systems, 24(4):4631–4640, 2023.

[52] Leo Breiman.
Random forests.
Machine Learning, 45(1):5–32, 2001.

[53] Corinna Cortes and Vladimir Vapnik.

Support-vector networks.
Machine Learning, 20(3):273–297, 1995.

[54] Tianqi Chen and Carlos Guestrin.
XGBoost: A scalable tree boosting system.
In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[55] Basem Almadani, Ekhlas Hashem, Raneem R. Attar, Farouq Aliyu, and Esam Al-Nahari.
Publish/subscribe-middleware-based intelligent transportation systems: Applications and challenges.
Applied Sciences, 15(12):6449, 2025.

[56] Jay Kreps, Jun Rao, and Jose Coraglia.
Kafka: A distributed messaging system for log processing.
In *Proceedings of the NetDB Workshop*, Athens, Greece, 2011.

[57] Silvia Mazzetto.
A review of urban digital twins integration, challenges, and future directions in smart city development.
Sustainability, 16(19):8337, 2024.

[58] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu.
Diffusion convolutional recurrent neural network: Data-driven traffic forecasting.
In *International Conference on Learning Representations (ICLR)*, 2018.

[59] Scott M. Lundberg and Su-In Lee.
A unified approach to interpreting model predictions.
In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 4765–4774, 2017.

[60] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bhavana Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee.
From local explanations to global understanding with explainable AI for trees.
Nature Machine Intelligence, 2(1):56–67, 2020.

[61] Zachary C. Lipton.
The mythos of model interpretability.
Queue, 16(3):31–57, 2018.

[62] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin.
"why should I trust you?": Explaining the predictions of any classifier.
In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

[63] Ramazan Seiitbek and Aivar Sakhipov.
Architectural patterns for urban incident management: A comparative analysis in astana's smart traffic ecosystem.
Unpublished manuscript in preparation; not publicly available; companion to the published IJICT microservices study (2026); generator rules summarised in thesis Appendix C (Astana semi-synthetic dataset), 2026.